

模型修正接口清单

本文档根据当前代码重新整理，范围仅包含 `src` 之外的接口，也就是：

- `app.py`
- `webapi/routers/*.py`

`src/l3` 下的 ODB/L3 接口不在本文档范围内。

1. 通用约定

1.1 成功返回包体

除个别服务函数本身返回 `null` 外，接口统一包裹为：

```
{
  "ok": true,
  "code": 200,
  "message": "success message",
  "data": {}
}
```

1.2 失败返回包体

```
{
  "ok": false,
  "code": 500,
  "message": "error message",
  "data": null,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "error message",
    "details": {}
  }
}
```

补充说明：

- 当前模型修正接口统一只返回两种业务码：
 - `200`：接口执行成功
 - `500`：接口执行失败，需要查看 `message` 与 `error.details`
- 部分原始异常如果本来属于 `400/404` 等类型，后端会统一折叠为 `500` 返回；原始状态码会放进 `error.details.original_status_code`。

1.3 求解器路径约定

- Abaqus 不通过单独接口配置。
- 后端会优先读取 `service_config.json` 中的配置。
- 如果请求体里显式传入 `abaqus`，则以请求值为准。

2. 系统接口

POST /init

接口含义：初始化数据库表。

功能说明：调用 `ensure_tables_exist()`，在开发阶段按当前代码创建所需表结构。

影响数据库：

- `db.py` 中模型修正链路相关表：建表/补表，仅确保表结构存在，不写入具体业务数据。

请求参数：无。

请求示例：

```
{}
```

返回 `data` 样式：

```
null
```

POST /project/config

接口含义：保存项目级模型修正配置。

功能说明：保存试验模型包络尺寸、FEM 模型包络尺寸、缩放系数和附加配置。节点匹配参数、自动平移/旋转估计等流程会复用这里的配置。

影响数据库：

- `t_mt_py_project_config`：写，按项目保存 `test_model_dims`、`fem_model_dims`、`coefficients` 和 `extra_json`。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>test_model_dims</code>	<code>object</code>	否	试验模型包络尺寸，包含 <code>x/y/z</code>
<code>fem_model_dims</code>	<code>object</code>	否	FEM 模型包络尺寸，包含 <code>x/y/z</code>
<code>coefficients</code>	<code>object</code>	否	缩放或换算系数字典
<code>extra_json</code>	<code>object</code>	否	额外配置

请求示例：

```
{
  "project_id": 101,
  "test_model_dims": {
    "x": 1200.0,
    "y": 300.0,
    "z": 180.0
  }
}
```

```
{,
  "fem_model_dims": {
    "x": 1180.0,
    "y": 305.0,
    "z": 175.0
  },
  "coefficients": {
    "length_scale": 1.0
  },
  "extra_json": {
    "note": "manual config"
  }
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "test_model_dims": {
    "x": 1200.0,
    "y": 300.0,
    "z": 180.0
  },
  "fem_model_dims": {
    "x": 1180.0,
    "y": 305.0,
    "z": 175.0
  },
  "coefficients": {
    "length_scale": 1.0
  },
  "extra_json": {
    "note": "manual config"
  }
}
```

POST /project/config/query

接口含义：查询项目级模型修正配置。

功能说明：读取 POST /project/config 已保存的项目配置；如果该项目还没有配置记录，则返回空尺寸和空字典。

影响数据库：

- t_mt_py_project_config：读，返回项目级配置。

请求参数：

参数名	类型	必填	说明
project_id	int	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "test_model_dims": {
    "x": 1200.0,
    "y": 300.0,
    "z": 180.0
  },
  "fem_model_dims": {
    "x": 1180.0,
    "y": 305.0,
    "z": 175.0
  },
  "coefficients": {
    "length_scale": 1.0
  },
  "extra_json": {
    "note": "manual config"
  }
}
```

3. 试验数据接口

POST /import/unv

接口含义：导入试验 UNV 文件。

功能说明：解析试验节点、单元、模态或静力结果，并写入试验相关表；同时会把测点信息写入 `t_mt_measuring_point_info`。

影响数据库：

- `t_mt_py_test_node`、`t_mt_py_test_element`：写，保存试验节点和单元。
- `t_mt_py_test_modal_frequency`、`t_mt_py_test_modal_shape`、`t_mt_py_test_modal_shape_real`、`t_mt_py_test_modal_shape_imag`：写，保存模态频率与振型。
- `t_mt_py_test_static_result`：写，保存静力结果。
- `t_mt_measuring_point_info`：写，保存或刷新测点信息。
- `t_mt_work_condition_project`：写，更新项目试验数据状态。

请求参数：

参数名	类型	必填	说明
file_path	string	是	UNV 文件路径

参数名	类型	必填	说明
project_id	int	否	项目 ID，代码层面允许为空，但业务上建议必传
file_id	int	否	文件 ID
clear_before_insert	bool	否	是否先清空该项目已有试验数据，默认 true

请求示例：

```
{
  "file_path": "D:/demo/test.unv",
  "project_id": 101,
  "clear_before_insert": true
}
```

返回 data 示例：

```
{
  "file_path": "D:/demo/test.unv",
  "result_kind": "modal",
  "cleared_before_insert": true,
  "test_node_count": 120,
  "measuring_point_count": 120,
  "test_element_count": 80,
  "test_mode_count": 6,
  "test_static_result_count": 0
}
```

POST /plot/modal_shape

接口含义：获取试验模态振型数据。

功能说明：读取试验节点、单元和模态振型，返回前端可直接用于可视化的结构化数据。

影响数据库：

- t_mt_py_test_node、t_mt_py_test_element：读，组装几何拓扑。
- t_mt_py_test_modal_frequency、t_mt_py_test_modal_shape：读，定位模态及振型内容。

请求参数：

参数名	类型	必填	说明
project_id	int	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 data 示例:

```
{
  "points": {
    "ids": [1, 2, 3],
    "position": [0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 0.0],
    "ItemSize": 3
  },
  "elements": {
    "type": 2,
    "index": [0, 1, 1, 2],
    "ItemSize": 2
  },
  "modal_shape": [
    {
      "order": 1,
      "frequency": "32.5",
      "unit": "Hz",
      "position": {
        "real": [0.0, 0.1, 0.0],
        "imag": [0.0, 0.0, 0.0]
      }
    }
  ]
}
```

POST /get/sensor_position

接口含义：获取测点位置。

功能说明：从 t_mt_measuring_point_info 中读取当前项目测点，返回测点标签、类型和坐标。

影响数据库：

- t_mt_measuring_point_info：读，返回测点标签、类型和坐标。

请求参数：

参数名	类型	必填	说明
project_id	int	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 data 示例:

```
[
  {
    "sensor_label": "wY1",
    "sensor_type": "位移",
    "sensor_pos": [0.0, 0.0, 0.0]
  },
  {
    "sensor_label": "wY2",
    "sensor_type": "位移",
    "sensor_pos": [1.0, 0.0, 0.0]
  }
]
```

POST /get/sensor_relative_error

接口含义：获取测点相对误差分布。

功能说明：读取测点表和 `t_mt_py_fem_analysis_error` 中最近一次静力对比结果，为每个测点补上误差值 `e_value`，用于前端按测点显示误差云图或列表。

影响数据库：

- `t_mt_measuring_point_info`：读，读取测点标签、类型和坐标。
- `t_mt_py_fem_analysis_error`：读，读取每个测点各分量的初始/更新相对误差，并为每个测点挑选一个当前展示值。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 `data` 示例：

```
[
  {
    "sensor_label": "wY1",
    "sensor_type": "位移",
    "sensor_pos": [0.0, 0.0, 0.0],
    "e_value": 0.031
  },
  {
    "sensor_label": "wY2",
    "sensor_type": "位移",
    "sensor_pos": [1.0, 0.0, 0.0],
    "e_value": null
  }
]
```

]

POST /get/deform_sensor_position

接口含义：获取变形后测点坐标。

功能说明：从 `t_mt_py_test_static_result` 读取指定静力位移结果，将 `ux/uy/uz` 乘以缩放系数后叠加到原测点坐标上，返回变形后位置。

影响数据库：

- `t_mt_py_test_static_result`：读，读取试验静力位移结果。
- `t_mt_measuring_point_info`：读，读取原始测点坐标。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>scale</code>	<code>float</code>	否	位移缩放系数，默认 <code>1.0</code>
<code>static_result_id</code>	<code>int</code>	否	静力结果主键 ID；传入时优先按此定位
<code>load_case_no</code>	<code>int</code>	否	载荷工况号
<code>result_no</code>	<code>int</code>	否	结果号

请求示例：

```
{
  "project_id": 101,
  "load_case_no": 1,
  "result_no": 1,
  "scale": 10.0
}
```

返回 `data` 示例：

```
[
  {
    "sensor_label": "wy1",
    "sensor_type": "位移",
    "sensor_pos": [0.01, 0.0, 0.0]
  }
]
```

POST /dump/vtk

接口含义：导出试验模态振型到 VTK 文件。

功能说明：基于试验模态结果写出 `.vtk` 文件。当前服务函数不返回文件信息，因此 `data` 为 `null`。

影响数据库：

- `t_mt_py_test_node`、`t_mt_py_test_element`：读，获取网格结构。
- `t_mt_py_test_modal_frequency`、`t_mt_py_test_modal_shape`：读，获取模态振型数据。

请求参数：

参数名	类型	必填	说明
<code>vtk_path</code>	<code>string</code>	是	输出 VTK 路径
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101,
  "vtk_path": "D:/demo/modal_shape.vtk"
}
```

返回 `data` 样式：

```
null
```

POST /dump/json

接口含义：导出试验模态振型到 JSON 文件。

功能说明：把 `/plot/modal_shape` 的结构化结果写成 JSON 文件。

影响数据库：

- `t_mt_py_test_node`、`t_mt_py_test_element`：读，获取网格结构。
- `t_mt_py_test_modal_frequency`、`t_mt_py_test_modal_shape`：读，获取模态振型数据。

请求参数：

参数名	类型	必填	说明
<code>json_path</code>	<code>string</code>	是	输出 JSON 路径
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101,
  "json_path": "D:/demo/modal_shape.json"
}
```

返回 `data` 示例：

```
{
  "json_path": "D:/demo/modal_shape.json"
}
```

4. FEM 基础导入与 INP 目录接口

POST /import/bdf

接口含义：导入 BDF 基础模型信息。

功能说明：解析材料、属性、边界等基础 FEM 表，适合 Nastran/BDF 流程的基础数据入库。

影响数据库：

- `t_mt_py_fem_material_overview`、`t_mt_py_fem_isotropic`：写，保存材料概览和各向同性材料参数。
- `t_mt_py_fem_property`、`t_mt_py_fem_shell_property`、`t_mt_py_fem_beam_property`：写，保存属性与截面数据。
- `t_mt_py_fem_boundary`：写，保存边界条件。

请求参数：

参数名	类型	必填	说明
<code>file_path</code>	<code>string</code>	是	BDF 文件路径
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>clear_before_insert</code>	<code>bool</code>	否	是否先清空 FEM 相关旧数据，默认 <code>true</code>

请求示例：

```
{
  "file_path": "D:/demo/model.bdf",
  "project_id": 101,
  "clear_before_insert": true
}
```

返回 data 示例：

```
{
  "file_path": "D:/demo/model.bdf",
  "cleared_before_insert": true
}
```

POST /import/inp/catalog

接口含义：导入 INP 并建立模型修正目录。

功能说明：解析 INP，建立参数定义、参数目标、支持修正物理量、集合能力目录，并可构建节点八叉树缓存。

影响数据库：

- `t_mt_py_fem_material_overview`、`t_mt_py_fem_isotropic`、`t_mt_py_fem_property`、`t_mt_py_fem_shell_property`、`t_mt_py_fem_beam_property`、`t_mt_py_fem_boundary`：写，刷新基础 FEM 目录。
- `t_mt_py_fem_parameter_definition`、`t_mt_py_fem_parameter_target`、`t_mt_py_fem_design_response_catalog`：写，保存参数定义、参数落点和设计响应目录。
- `t_mt_py_fem_supported_quantity`、`t_mt_py_fem_quantity_set_capability`：写，保存修正量和集合能力目录。
- `t_mt_py_fem_node_octree_cache`：写，保存节点八叉树缓存元数据。
- `t_mt_py_fem_selected_parameter`：写，清理旧优化参数，避免目录变更后残留脏数据。

请求参数：

参数名	类型	必填	说明
<code>file_path</code>	<code>string</code>	是	INP 文件路径
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>clear_before_insert</code>	<code>bool</code>	否	是否清空该项目旧 FEM/目录数据，默认 <code>true</code>
<code>build_octree</code>	<code>bool</code>	否	是否构建节点八叉树缓存，默认 <code>true</code>
<code>force_rebuild_octree</code>	<code>bool</code>	否	是否强制重建八叉树缓存，默认 <code>false</code>

请求示例：

```
{
  "file_path": "D:/demo/model.inp",
  "project_id": 101,
  "clear_before_insert": true,
  "build_octree": true,
  "force_rebuild_octree": false
}
```

返回 `data` 示例：

```
{
  "file_path": "D:/demo/model.inp",
  "project_id": 101,
  "material_count": 2,
  "isotropic_material_count": 2,
  "property_count": 3,
  "shell_property_count": 1,
  "beam_property_count": 0,
  "boundary_count": 0,
  "parameter_definition_count": 8,
  "parameter_target_count": 8,
  "design_response_count": 4,
  "supported_quantity_count": 2,
  "quantity_set_capability_count": 5,
}
```

```
"instance_count": 1,
"node_count": 1200,
"octree_cache_path": "D:/demo/cache/project_101/model.node_octree.npz",
"diagnostics": 0
}
```

POST /catalog/inp

接口含义：查询已导入的 INP 全量目录信息。

功能说明：读取数据库中的 INP 目录，不重新解析源文件。

影响数据库：

- `t_mt_py_fem_parameter_definition`、`t_mt_py_fem_parameter_target`、`t_mt_py_fem_design_response_catalog`：读，返回参数和响应目录。
- `t_mt_py_fem_supported_quantity`、`t_mt_py_fem_quantity_set_capability`、`t_mt_py_fem_selected_parameter`：读，返回可选修正量、集合能力和已创建优化参数。
- `t_mt_py_fem_node_octree_cache`：读，返回已缓存的 INP/Octree 元数据。
- `t_mt_py_fem_node_match`、`t_mt_py_fem_dof_match`、`t_mt_py_fem_response_catalog`、`t_mt_py_fem_modal_result`、`t_mt_py_fem_modal_correlation`：读，补充返回关联流程统计信息。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 data 示例：

```
{
  "project_id": 101,
  "parameter_definitions": [],
  "parameter_targets": [],
  "design_responses": [],
  "supported_quantities": [],
  "quantity_set_capabilities": [],
  "optimization_parameters": [],
  "octree_cache": null,
  "node_match_count": 0,
  "dof_match_count": 0,
  "response_catalog_count": 0,
  "fem_mode_count": 0,
  "modal_correlation_count": 0
}
```

POST /catalog/inp/parameter-options

接口含义：查询 INP 参数可选项。

功能说明：按照“修正物理量 + 支持等级 + 可选集合”返回参数选择面板所需数据。当前支持量以代码里的 E、H 为主。

影响数据库：

- `t_mt_py_fem_supported_quantity`：读，读取已启用修正量。
- `t_mt_py_fem_quantity_set_capability`：读，读取每个修正量对应的可选集合和支持等级。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 data 示例：

```
[
  {
    "parameter_name": "E",
    "description": "杨氏模量",
    "level": "GLOBAL",
    "sets": ["SET_A", "SET_B"]
  },
  {
    "parameter_name": "H",
    "description": "壳厚",
    "level": "LOCAL",
    "sets": ["SHELL_SET"]
  }
]
```

POST /tools/inp/tree

接口含义：查看 INP 树状结构。

功能说明：用于调试和人工核对 INP 结构。当前底层实现主要把结构打印到服务端控制台，API 返回值通常为 `null`。

影响数据库：

- 无。该接口只解析 INP 文件并输出结构，不直接读写数据库。

请求参数：

参数名	类型	必填	说明
<code>file_path</code>	<code>string</code>	是	INP 文件路径
<code>show_labels</code>	<code>bool</code>	否	是否显示节点/单元标签摘要，默认 <code>true</code>
<code>max_labels</code>	<code>int</code>	否	每类标签最多显示数量，默认 <code>8</code>

请求示例：

```
{
  "file_path": "D:/demo/model.inp",
  "show_labels": true,
  "max_labels": 8
}
```

返回 `data` 样式：

```
null
```

POST /import/fem/static/from_project_result

接口含义：把项目结果工作区中的静力位移结果导入到 FEM 静力结果表。

功能说明：从项目工作区指定 `result_group`、`step`、`frame` 和实例列表中提取位移场，转换成模型修正链路使用的 FEM 静力结果记录，并写入 `t_mt_py_fem_static_result`。

影响数据库：

- `t_mt_py_fem_static_result`：写，保存按 `load_case_no` 归档的 FEM 静力结果。
- 项目工作区 `manifest.db` / `HDF5`：读，从已有项目结果中抽取位移场。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>result_group</code>	<code>string</code>	是	项目结果组名称
<code>load_case_no</code>	<code>int</code>	否	导入后的静力工况号，默认 <code>1</code>
<code>step</code>	<code>string</code>	否	指定分析步；不传时由后端选择
<code>frame</code>	<code>int</code>	否	指定帧号；不传时由后端选择
<code>instances</code>	<code>string[]</code>	否	只导入指定实例
<code>overwrite</code>	<code>bool</code>	否	是否先删除该工况旧结果，默认 <code>true</code>

请求示例：

```
{
  "project_id": 101,
  "result_group": "default_result",
  "load_case_no": 1,
  "step": "StaticStep",
  "frame": 0,
  "instances": ["PART-1-1"],
  "overwrite": true
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "load_case_no": 1,
  "row_count": 1200,
  "result_group": "default_result",
  "workspace": "D:/demo/model/101",
  "step_name": "StaticStep",
  "frame_idx": 0,
  "instances": ["PART-1-1"],
  "overwrite": true
}
```

5. 参数定义与模型修正接口

POST /add/response

接口含义: 手工新增一个修正响应定义。

功能说明: 补充模型修正中使用的响应项, 写入响应类型、自由度、散度和分析步信息。

影响数据库:

- `t_mt_py_fem_manual_response`: 写, 保存手工补充的修正响应定义。

请求参数:

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>type</code>	<code>string</code>	是	响应类型
<code>scatter</code>	<code>float</code>	是	响应散度
<code>dof</code>	<code>string</code>	是	自由度, 例如 <code>ux</code>
<code>step</code>	<code>string</code>	否	分析步名称

请求示例:

```
{
  "project_id": 101,
  "type": "DISPLACEMENT",
  "scatter": 0.05,
  "dof": "UX",
  "step": "Step-1"
}
```

返回 data 示例：

```
{
  "project_id": 101,
  "type": "DISPLACEMENT",
  "scatter": 0.05,
  "dof": "UX",
  "step": "Step-1"
}
```

POST /optimization/parameter/create

接口含义：创建优化参数。

功能说明：从已导入的 INP 参数能力目录中，按“优化参数类型 + 对应集合名称”生成正式参与优化的参数记录；selection_mode 未传时由后端根据集合能力自动推断。

补充说明：

- 当前最小必填输入为：project_id、优化参数类型、set_name。
- 优化参数类型当前建议直接传 quantity_code；candidate_code 仅作为兼容别名保留。
- set_name 支持单个字符串或字符串数组。
- 传多个 set_name 时，会按每个集合分别创建参数。

影响数据库：

- t_mt_py_fem_quantity_set_capability：读，校验目标集合是否支持当前修正量及选择模式。
- t_mt_py_fem_selected_parameter：读写，检查目标集合是否与现有参数重叠，并写入最终优化参数记录。

请求参数：

参数名	类型	必填	说明
project_id	int	是	项目 ID
set_name	string \ \nstring[]	是	目标集合名，支持单个或多个
quantity_code	string	是	优化参数类型/修正量编码，目前主要为 E、H
lower	float	是	参数下界

参数名	类型	必填	说明
upper	float	是	参数上界
candidate_code	string	否	候选参数编码，兼容旧调用方式
prob_id	int	否	问题 ID，默认 0
selection_mode	string	否	选择模式；未传时通常会自动解析为 GLOBAL 或 LOCAL
parameter_name	string	否	自定义参数名
scatter	float	否	参数散度
description	string	否	备注说明，默认空字符串
set_type	string	否	集合类型，如 ELSET
set_scope	string	否	集合范围，如 PART、ASSEMBLY
instance_name	string	否	实例名
part_name	string	否	零件名

请求示例：

```
{
  "project_id": 101,
  "set_name": "SET_A",
  "quantity_code": "E",
  "lower": 100000.0,
  "upper": 300000.0,
  "prob_id": 0,
  "parameter_name": "E@SET_A"
}
```

返回 data 示例：

```
{
  "project_id": 101,
  "parameter_group_name": "E@SET_A",
  "quantity_code": "E",
  "selection_mode": "GLOBAL",
  "set_name": "SET_A",
  "set_type": "ELSET",
  "set_scope": "PART",
  "instance_name": null,
  "part_name": "PART-1",
  "lower": 100000.0,
  "upper": 300000.0,
  "prob_id": 0,
  "scatter": 0.15,
  "description": "",
}
```

```
"created_parameter_count": 1,
"created_parameters_preview": [
  {
    "parameter_name": "E@SET_A",
    "element_label": null,
    "current_value": 210000.0
  }
]
}
```

POST /optimization/bayesian/run

接口含义：执行贝叶斯模型修正。

功能说明：读取灵敏度矩阵、当前参数、目标响应并执行迭代更新；可按需重新调用 Abaqus；支持最终结果写回云图。

影响数据库：

- `t_mt_py_fem_analysis_run`、`t_mt_py_fem_response_def`、`t_mt_py_fem_parameter_def`、`t_mt_py_fem_sensitivity_result`：读，加载当前批次灵敏度矩阵及行列元数据。
- `t_mt_py_fem_selected_parameter`、`t_mt_py_fem_response_catalog`：读，解析参与修正的参数与响应信息。
- `t_mt_py_fem_tracking_iteration`、`t_mt_py_fem_response_difference`、`t_mt_py_fem_tracking_value`、`t_mt_py_fem_parameter_variation`：写，保存迭代过程跟踪结果。
- `t_mt_py_fem_model_update_static_result`：写，保存本轮修正后的静力结果快照。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>input_inp</code>	<code>string</code>	是	输入 INP 路径
<code>target_responses</code>	<code>any</code>	是	目标响应向量或键值映射
<code>batch_no</code>	<code>int</code>	否	批次号，默认 1
<code>parameter_scatter</code>	<code>any</code>	否	参数散度，可为标量、数组或映射
<code>response_scatter</code>	<code>any</code>	否	响应散度，可为标量、数组或映射
<code>save_results</code>	<code>bool</code>	否	是否保存迭代文件，默认 <code>true</code>
<code>odb_id</code>	<code>string</code>	否	已加载 ODB 标识
<code>base_url</code>	<code>string</code>	否	L3 服务地址
<code>workspace</code>	<code>string</code>	否	已构建好的 workspace 路径
<code>odb_path</code>	<code>string</code>	否	ODB 文件路径

参数名	类型	必填	说明
step	string	否	结果步名
instances	string[]	否	实例过滤列表
field_prefix	string	否	灵敏度字段前缀，默认 d_u_
response_component	string	否	响应分量
position	string	否	结果位置
aggregation	string	否	聚合方式，默认 max_abs
frame	int	否	帧号，默认 0
iterations	int	否	迭代次数，默认 1
exit_diff_percent	float	否	提前停止阈值
damping	float	否	贝叶斯阻尼，默认 1e-8
step_scale	float	否	更新步长缩放，默认 1.0
lower_bound	any	否	参数下界
upper_bound	any	否	参数上界
keep_raw	bool	否	是否保留原始中间文件，默认 false
timeout	int	否	ODB/workspace 处理超时秒数，默认 60
job_name	string	否	求解作业名
cpus	int	否	Abaqus CPU 数
interactive	bool	否	是否以 interactive 方式运行，默认 true
run_solver	bool	否	是否执行求解，默认 false
timeout_sec	int	否	求解超时秒数
extra_args	string[]	否	额外命令行参数
write_cloud_result	bool	否	是否写入结果云图，默认 false
cloud_result_group	string	否	云图结果组名
cloud_step_name	string	否	云图步名，默认 BayesianUpdate
cloud_field_name	string	否	云图字段名，默认 PARAMETER_CLOUD
cloud_value_mode	string	否	云图写值模式，默认 updated_value

请求示例：

```
{
  "project_id": 101,
  "input_inp": "D:/demo/model.inp",
  "target_responses": {
    "MODE_FREQ:1": 32.5,
    "NODE_DOF:PART-1-1:1001:U1:1:UX": 0.12
  },
  "parameter_scatter": 0.15,
  "response_scatter": 0.05,
  "iterations": 2,
  "run_solver": false
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "batch_no": "1",
  "input_inp": "D:/demo/model.inp",
  "output_dir": "D:/demo/model_bayesian",
  "save_results": true,
  "iterations": 2,
  "requested_iterations": 2,
  "stopped_early": false,
  "exit_diff_percent": null,
  "final_updated_inp": "D:/demo/model_bayesian/iter_2/model.inp",
  "history_dir": "D:/demo/model_bayesian/history",
  "history_html": "D:/demo/model_bayesian/history/overview.html",
  "cloud_result": null,
  "final_static_output": {},
  "iteration_dirs": [
    "D:/demo/model_bayesian/iter_1",
    "D:/demo/model_bayesian/iter_2"
  ]
}
```

配置来源:

- `output_dir` 不再从请求体传入, 默认读取 `service_config.json` 的 `APP_BAYESIAN_OUTPUT_DIR`。
- `abaqus` 不再从请求体传入, 默认读取 `service_config.json` 的 `APP_ABAQUS_CMD`。
- `python3` 不再从请求体传入, 默认读取 `service_config.json` 的 `APP_PYTHON3_CMD`。

异步提交:

`/optimization/bayesian/run` 支持和灵敏度任务相同的后台提交方式。请求体增加:

```
{
  "async_submit": true
}
```

接口会立即返回:

```
{
  "task_id": "xxxx",
  "task_type": "optimization.bayesian.run",
  "status": "submitted"
}
```

状态查询接口：

```
GET /optimization/bayesian/tasks?task_id=...
```

状态值包括：

- submitted：任务已提交，等待线程池执行。
- running：正在执行 Bayesian 迭代，data.progress.current_iteration 表示当前跑到第几轮，data.progress.total_iterations 表示总轮数。
- succeeded：执行成功，data.result 是压缩后的 Bayesian 结果摘要。
- failed：执行失败，data.error 包含错误信息。
- aborted：服务重启后恢复任务状态时，原来未完成的任务会被标记为中断。

POST /optimization/bayesian/check

接口含义：基于文本矩阵校验贝叶斯更新过程。

功能说明：不依赖 ODB/workspace，直接从文本文件读取灵敏度矩阵、模型响应和目标响应，验证贝叶斯公式和参数更新逻辑。

影响数据库：

- 无。该接口只读取文本文件和请求体数据，不直接读写数据库。

请求参数：

参数名	类型	必填	说明
sensitivity_matrix.file_path	string	是	灵敏度矩阵文本路径
sensitivity_matrix.row_start	int	是	起始行号，从 1 开始
sensitivity_matrix.row_count	int	是	读取行数
model_response.file_path	string	是	模型响应文本路径
model_response.row	int	是	响应所在行号
target_response.file_path	string	是	目标响应文本路径
target_response.row	int	是	响应所在行号
parameter_names	string[]	是	参数名列表
sensitivity_matrix.col_start	int	否	起始列号，默认 1
model_response.col_start	int	否	起始列号，默认 1

参数名	类型	必填	说明
<code>target_response.col_start</code>	<code>int</code>	否	起始列号，默认 1
<code>parameter_scatter</code>	<code>any</code>	否	参数散度
<code>response_scatter</code>	<code>any</code>	否	响应散度
<code>input_inp</code>	<code>string</code>	否	INP 路径；未传 <code>parameter_values</code> 时必需
<code>parameter_values</code>	<code>any</code>	否	当前参数值
<code>damping</code>	<code>float</code>	否	阻尼，默认 1e-8
<code>step_scale</code>	<code>float</code>	否	步长系数，默认 1.0
<code>lower_bound</code>	<code>any</code>	否	参数下界
<code>upper_bound</code>	<code>any</code>	否	参数上界
<code>output_dir</code>	<code>string</code>	否	输出目录
<code>case_name</code>	<code>string</code>	否	案例名，默认 <code>bayesian_text_check</code>

请求示例：

```
{
  "sensitivity_matrix": {
    "file_path": "D:/demo/S.txt",
    "row_start": 1,
    "row_count": 2,
    "col_start": 1
  },
  "model_response": {
    "file_path": "D:/demo/r_model.txt",
    "row": 1,
    "col_start": 1
  },
  "target_response": {
    "file_path": "D:/demo/r_target.txt",
    "row": 1,
    "col_start": 1
  },
  "parameter_names": ["P1", "P2"],
  "parameter_values": [1.0, 2.0],
  "parameter_scatter": [0.15, 0.15],
  "response_scatter": [0.05, 0.05],
  "output_dir": "D:/demo/bayesian_check"
}
```

返回 data 示例：

```
{
  "case_name": "bayesian_text_check",
  "input_inp": "D:/demo/model.inp",
  "sensitivity_matrix_file": "D:/demo/S.txt",
  "model_response_file": "D:/demo/r_model.txt",
  "target_response_file": "D:/demo/r_target.txt",
  "parameter_names": ["P1", "P2"],
  "parameter_values": [1.0, 2.0],
  "parameter_scatter": [0.15, 0.15],
  "response_scatter": [0.05, 0.05],
  "sensitivity_matrix": [[0.1, 0.2], [0.3, 0.4]],
  "model_response": [1.2, 0.8],
  "target_response": [1.0, 1.0],
  "bayesian": {
    "p_new": [1.05, 1.94]
  }
}
```

6. 节点匹配、空间变换与一致性评价接口

POST /pair/node_point

别名：POST /match/nodes

接口含义：试验节点与 FEM 节点匹配。

功能说明：基于试验节点坐标和 FEM 八叉树缓存执行最近点匹配，可自动平移、自动旋转，也可手工指定变换。

影响数据库：

- `t_mt_py_fem_node_octree_cache`：读，获取项目节点八叉树缓存和源 INP 元数据。
- `t_mt_py_test_node`：读，读取试验节点坐标。
- `t_mt_py_fem_node_match`：写，保存节点匹配结果。
- `t_mt_py_fem_dof_match`、`t_mt_py_fem_response_catalog`、`t_mt_py_fem_modal_correlation`：写，`overwrite=true` 时清理依赖旧节点匹配的下游结果。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>max_distance</code>	<code>float</code>	否	最大允许匹配距离
<code>overwrite</code>	<code>bool</code>	否	是否覆盖旧匹配结果，默认 <code>true</code>
<code>auto_translate</code>	<code>bool</code>	否	是否自动平移，默认 <code>true</code>
<code>translation</code>	<code>float[3]</code>	否	手工平移向量 <code>[x, y, z]</code>
<code>rotation.center</code>	<code>float[3]</code>	否	旋转中心
<code>rotation.axis</code>	<code>float[3]</code>	否	旋转轴

参数名	类型	必填	说明
rotation.angle_deg	float	否	旋转角度，单位度

请求示例:

```
{
  "project_id": 101,
  "max_distance": 5.0,
  "auto_translate": true,
  "rotation": {
    "center": [0.0, 0.0, 0.0],
    "axis": [0.0, 0.0, 1.0],
    "angle_deg": 1.5
  }
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "tested_points": 120,
  "matched_points": 118,
  "transform": {
    "mode": "icp",
    "translation": [0.1, 0.0, 0.0],
    "rotation": {
      "center": [0.0, 0.0, 0.0],
      "axis": [0.0, 0.0, 1.0],
      "angle_deg": 1.5
    },
  },
  "fit": {
    "iterations": 12,
    "rmse": 0.002,
    "pair_count": 120
  },
  "max_distance": null,
  "matches_preview": [
    {
      "test_node_id": "1",
      "instance_name": "PART-1-1",
      "part_name": "PART-1",
      "fem_node_label": 1001,
      "fem_coord": [0.0, 0.0, 0.0],
      "distance": 0.001,
      "x_offset": 0.001,
      "y_offset": 0.0,
      "z_offset": 0.0
    }
  ],
  "octree_cache_path": "D:/demo/cache/project_101/model.node_octree.npz"
}
```


POST /match/nodes/params

接口含义：获取节点匹配参数上下文。

功能说明：根据项目级配置和测点/FEM 模型包络尺寸，返回节点匹配界面可直接使用的推荐参数，包括外轮廓类型、试验模型尺寸、FEM 模型尺寸、推荐容差和最大节点点位距离。若项目配置里缺少试验模型尺寸，后端会尝试根据测点坐标自动回填。

影响数据库：

- `t_mt_py_project_config`：读，读取项目级 `test_model_dims`、`fem_model_dims` 和附加配置；必要时回写自动补出的试验模型尺寸。
- `t_mt_measuring_point_info`：读，当项目配置中缺少试验模型尺寸时，使用测点坐标计算试验模型包络尺寸。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 `data` 示例：

```
{
  "project_id": 101,
  "outer_contour_type": "axis_aligned_bbox",
  "test_model": {
    "dims": {
      "x": 1200.0,
      "y": 300.0,
      "z": 180.0
    },
    "model_size": 1200.0
  },
  "fem_model": {
    "dims": {
      "x": 1180.0,
      "y": 305.0,
      "z": 175.0
    },
    "model_size": 1180.0
  },
  "tolerance": 0.00118,
  "maximum_node_point_distance": 60.0
}
```

POST /get/pair_node_point_result

接口含义：获取节点匹配结果。

功能说明：根据已保存的节点匹配关系，返回保持一一对应关系的测点名数组和 FEM 节点坐标数组。

影响数据库：

- `t_mt_py_fem_node_match`：读，读取节点匹配关系。
- `t_mt_py_test_node`：读，补充试验节点信息。
- `t_mt_measuring_point_info`：读，补充测点名称。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 data 示例：

```
{
  "sensor_name": ["WY1", "WY2"],
  "node_xyz": [
    [0.1, 0.2, 0.3],
    [0.5, 0.3, 0.5]
  ]
}
```

POST /correlation/evaluate

接口含义：执行一致性评价并落库。

功能说明：基于试验静力结果、FEM 静力结果和节点匹配关系，计算 DAC、DSF，并将分析误差写入 `t_mt_py_fem_analysis_error`，同时把 DAC/DSF 写入 `t_mt_py_fem_dac_dsf`。

影响数据库：

- `t_mt_py_test_static_result`、`t_mt_py_fem_static_result`、`t_mt_py_fem_node_match`：读，加载试验结果、FEM 结果和节点映射。
- `t_mt_py_fem_analysis_error`：写，保存逐点逐分量误差明细。
- `t_mt_py_fem_dac_dsf`：写，保存 DAC/DSF 汇总结果。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

参数名	类型	必填	说明
load_case_no	int	否	指定载荷工况号
result_no	int	否	指定试验结果号
components	string[]	否	参与评价的分量, 如 ["ux", "uy", "uz"]
include_rotations	bool	否	是否包含转角分量, 默认 false

请求示例:

```
{
  "project_id": 101,
  "load_case_no": 1,
  "result_no": 1,
  "components": ["ux", "uy", "uz"],
  "include_rotations": false
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "load_case_no": 1,
  "result_no": 1,
  "components": ["ux", "uy", "uz"],
  "aligned_point_count": 30,
  "value_count": 90,
  "component_value_counts": {
    "ux": 30,
    "uy": 30,
    "uz": 30
  },
  "analysis_error_row_count": 90,
  "analysis_error_preview": [
    {
      "load_case_no": 1,
      "result_no": 1,
      "point_no": "1",
      "node_no": "PART-1-1:1001",
      "component_name": "ux",
      "point_value": 0.12,
      "initial_node_value": 0.1,
      "initial_relative_error": 16.67,
      "initial_abs_error": 0.02,
      "sensor_type_id": null
    }
  ],
  "dac": 98.5,
  "dsf": 1.02,
  "extra": {}
}
```

POST /transform/operation

接口含义：保存空间变换矩阵。

功能说明：将 `fem` 或 `test` 的 4x4 变换矩阵保存到数据库，供后续对齐流程复用。

影响数据库：

- `t_mt_py_fem_transform_operation`：写，保存空间变换矩阵。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>type</code>	<code>string</code>	是	变换类型，约定为 <code>fem</code> 或 <code>test</code>
<code>matrix4</code>	<code>float[4][4]</code>	是	4x4 齐次变换矩阵

请求示例：

```
{
  "project_id": 101,
  "type": "fem",
  "matrix4": [
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 1, 2],
    [0, 0, 0, 1]
  ]
}
```

返回 `data` 示例：

```
{
  "project_id": 101,
  "type": "fem",
  "matrix4": [
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 1, 2],
    [0, 0, 0, 1]
  ]
}
```

POST /transform/auto_info

接口含义：获取自动变换信息。

功能说明：读取已保存的空间变换记录。若不传 `type`，返回该项目最近一次保存的变换；传入 `type` 时按类型精确查询。

影响数据库：

- `t_mt_py_fem_transform_operation`: 读, 查询最近一次或指定类型的空间变换矩阵。

请求参数:

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>type</code>	<code>string</code>	否	变换类型, <code>fem</code> 或 <code>test</code>

请求示例:

```
{
  "project_id": 101,
  "type": "fem"
}
```

返回 `data` 示例:

```
{
  "type": "fem",
  "matrix4": [
    [1, 0, 0, 0],
    [0, 1, 0, 0],
    [0, 0, 1, 2],
    [0, 0, 0, 1]
  ]
}
```

7. 灵敏度接口

POST /sensitivity/run_and_store

接口含义: 直接运行已有的灵敏度分析 INP, 并将解析后的灵敏度矩阵存库。

功能说明: 调用方传入已经准备好的灵敏度 `inp` 文件, 服务端执行 Abaqus 求解, 生成 `odb` 后构建 workspace, 提取归一化灵敏度矩阵并保存到数据库。求解成功且 workspace 构建完成后, 会删除常见 Abaqus 过程文件, 只保留 `odb`、`inp` 与 workspace。

影响数据库:

- `t_mt_py_fem_analysis_run`: 写, 创建或更新当前批次灵敏度运行记录。
- `t_mt_py_fem_response_def`、`t_mt_py_fem_parameter_def`、`t_mt_py_fem_sensitivity_result`: 写, 保存灵敏度矩阵的响应定义、参数定义和矩阵值。
- `t_mt_py_fem_selected_parameter`: 写, 清空旧记录并按当前 `inp` 临时重建本次运行使用的参数映射。
- 外部结果服务: 可选调用 `/api/odb/{odb_id}/results/external-field`, 将灵敏度结果映射回单元/节点后写入 ODB 外部结果。

请求参数:

参数名	类型	必填	说明
project_id	int	是	项目 ID
input_inp	string	是	已准备好的灵敏度分析 INP 路径
output_dir	string	是	计算输出目录
step	string	是	结果步名，不再自动猜测
instances	string[]	是	参与提取的实例列表，不再自动猜测
field_prefix	string	是	灵敏度字段前缀，例如 d_u_
response_component	string	是	响应分量，例如 u1、u2
position	string	是	结果位置，例如 NODAL、ELEMENT_NODAL
batch_no	string	否	批次号，默认 "1"
aggregation	string	否	聚合方式，默认 max_abs
frame	int	否	帧号，默认 0
abaqus	string	否	Abaqus 命令
python3	string	否	Python 解释器
keep_raw	bool	否	是否保留 workspace 构建中的原始中间文件，默认 false
timeout	int	否	workspace/ODB 处理超时秒数，默认 60
job_name	string	否	作业名
cpus	int	否	CPU 数
interactive	bool	否	是否 interactive 运行，默认 true
timeout_sec	int	否	求解超时秒数
extra_args	string[]	否	额外命令参数
cleanup_process_files	bool	否	求解成功后是否删除 .com/.prt/.pmg/.pes/.par/.msg/.sta/.dat，默认 true
write_cloud_result	bool	否	是否把灵敏度结果按 external-field 形式写回 ODB，默认 false
cloud_result_group	string	否	云图结果组名，未传时按批次自动生成
cloud_step_name	string	否	写回 ODB 的结果步名，默认 Sensitivity
cloud_field_name	string	否	写回 ODB 的字段名，默认 SENSITIVITY_CLOUD

请求示例：

```
{
  "project_id": 101,
  "batch_no": "1",
  "input_inp": "D:/demo/out/model_sensitivity.inp",
  "output_dir": "D:/demo/out",
  "step": "Step-1",
  "instances": ["PART-1-1"],
```

```

"field_prefix": "d_U_",
"response_component": "U1",
"position": "NODAL",
"write_cloud_result": true,
"cloud_result_group": "sensitivity_rg_1",
"cloud_step_name": "Sensitivity",
"cloud_field_name": "SENSITIVITY_CLOUD"
}

```

返回 data 示例:

```

{
  "analysis_run_id": 12,
  "project_id": 101,
  "batch_no": "1",
  "case_name": "model_sensitivity",
  "response_names": ["PART-1-1|U|U1|NODAL|10"],
  "parameter_names": ["T1", "T2"],
  "response_count": 1,
  "parameter_count": 2,
  "point_count": 2,
  "input_inp": "D:/demo/out/model_sensitivity.inp",
  "analysis_inp": "D:/demo/out/model_sensitivity.inp",
  "odb_path": "D:/demo/out/model_sensitivity.odb",
  "workspace": "D:/demo/out/model_sensitivity_workspace",
  "step": "Step-1",
  "instances": ["PART-1-1"],
  "aggregation": "max_abs",
  "frame": 0,
  "field_prefix": "d_U_",
  "solver": {
    "ok": true
  },
  "generated_files": {
    "analysis_inp": "D:/demo/out/model_sensitivity.inp"
  },
  "temp_selected_parameters": {
    "selected_parameter_count": 2,
    "cleared_count": 2
  },
  "deleted_process_files": [
    "D:/demo/out/model_sensitivity.com",
    "D:/demo/out/model_sensitivity.dat"
  ],
  "cloud_result": {
    "odb_id": "odb-demo",
    "result_group": "sensitivity_rg_1",
    "step": "Sensitivity",
    "field": "SENSITIVITY_CLOUD",
    "components": ["T1", "T2"],
    "frame_count": 1
  }
}

```

POST /sensitivity/generate_run_and_store

接口含义：从数据库读取优化参数和设计响应，生成灵敏度分析 INP 后执行求解并存库。

功能说明：服务端读取 `t_mt_py_fem_selected_parameter` 和 `t_mt_py_fem_design_response_catalog`，调用项目侧提供的灵敏度 `inp` 生成函数，得到新的分析 `inp` 后复用 `/sensitivity/run_and_store` 的流程执行求解、解析 `odb`、存储矩阵并清理过程文件。

当前约束：接口已预留并完成数据库读取、流程编排与返回结构，实际“根据参数与设计响应生成灵敏度 `inp`”的项目侧函数需单独接入。

影响数据库：

- `t_mt_py_fem_selected_parameter`：读，读取优化参数定义与当前值。
- `t_mt_py_fem_design_response_catalog`：读，读取设计响应定义。
- `t_mt_py_fem_analysis_run`：写，创建或更新当前批次灵敏度运行记录。
- `t_mt_py_fem_response_def`、`t_mt_py_fem_parameter_def`、`t_mt_py_fem_sensitivity_result`：写，保存灵敏度矩阵的响应定义、参数定义和矩阵值。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>input_inp</code>	<code>string</code>	是	原始 INP 路径
<code>output_dir</code>	<code>string</code>	是	生成与计算输出目录
<code>step</code>	<code>string</code>	是	结果步名，不再自动猜测
<code>instances</code>	<code>string[]</code>	是	参与提取的实例列表，不再自动猜测
<code>field_prefix</code>	<code>string</code>	是	灵敏度字段前缀，例如 <code>d_u_</code>
<code>response_component</code>	<code>string</code>	是	响应分量，例如 <code>u1</code> 、 <code>u2</code>
<code>position</code>	<code>string</code>	是	结果位置，例如 <code>NODAL</code> 、 <code>ELEMENT_NODAL</code>
<code>batch_no</code>	<code>string</code>	否	批次号，默认 <code>"1"</code>
<code>aggregation</code>	<code>string</code>	否	聚合方式，默认 <code>max_abs</code>
<code>frame</code>	<code>int</code>	否	帧号，默认 <code>0</code>
<code>abaqus</code>	<code>string</code>	否	Abaqus 命令
<code>python3</code>	<code>string</code>	否	Python 解释器
<code>keep_raw</code>	<code>bool</code>	否	是否保留 workspace 构建中的原始中间文件，默认 <code>false</code>
<code>timeout</code>	<code>int</code>	否	workspace/ODB 处理超时秒数，默认 <code>60</code>
<code>job_name</code>	<code>string</code>	否	作业名
<code>cpus</code>	<code>int</code>	否	CPU 数
<code>interactive</code>	<code>bool</code>	否	是否 interactive 运行，默认 <code>true</code>
<code>timeout_sec</code>	<code>int</code>	否	求解超时秒数
<code>extra_args</code>	<code>string[]</code>	否	额外命令参数

参数名	类型	必填	说明
<code>cleanup_process_files</code>	<code>bool</code>	否	求解成功后是否删除 <code>.com/.prt/.pmg/.pes/.par/.msg/.sta/.dat</code> ，默认 <code>true</code>

请求示例：

```
{
  "project_id": 101,
  "batch_no": "1",
  "input_inp": "D:/demo/model.inp",
  "output_dir": "D:/demo/out",
  "step": "Step-1",
  "instances": ["PART-1-1"],
  "field_prefix": "d_U_",
  "response_component": "U1",
  "position": "NODAL"
}
```

返回 `data` 示例：

```
{
  "analysis_run_id": 12,
  "project_id": 101,
  "batch_no": "1",
  "input_inp": "D:/demo/model.inp",
  "analysis_inp": "D:/demo/out/model_generated_sensitivity.inp",
  "generation_source": "project_db",
  "loaded_parameter_count": 8,
  "loaded_design_response_count": 2,
  "generated_files": {
    "design_parameter_inp": "D:/demo/out/designParameter.inp",
    "analysis_inp": "D:/demo/out/model_generated_sensitivity.inp"
  },
  "odb_path": "D:/demo/out/model_generated_sensitivity.odb",
  "workspace": "D:/demo/out/model_generated_sensitivity_workspace",
  "deleted_process_files": [
    "D:/demo/out/model_generated_sensitivity.com",
    "D:/demo/out/model_generated_sensitivity.dat"
  ]
}
```

POST `/sensitivity/calculate_and_store`

接口含义：兼容旧的 DSA 灵敏度计算与存储接口。

功能说明：保留原有“可按需生成并执行 Abaqus 灵敏度工况，再从 ODB 或 workspace 中提取灵敏度矩阵并存库”的调用方式。新项目建议优先使用 `/sensitivity/run_and_store` 与 `/sensitivity/generate_run_and_store`，将“已有灵敏度 `inp`”和“从数据库生成灵敏度 `inp`”两类流程分开调用。

POST /sensitivity/stored/table

接口含义：读取已存储的灵敏度表格点位数据。

功能说明：将灵敏度矩阵按点表形式展开，适合前端表格显示。

影响数据库：

- t_mt_py_fem_analysis_run、t_mt_py_fem_response_def、t_mt_py_fem_parameter_def、t_mt_py_fem_sensitivity_result：读，读取已存储灵敏度矩阵并展开为点表。

请求参数：

参数名	类型	必填	说明
project_id	int	是	项目 ID
batch_no	string	否	批次号，默认 "1"

请求示例：

```
{
  "project_id": 101,
  "batch_no": "1"
}
```

返回 data 示例：

```
{
  "analysis_run_id": 12,
  "project_id": 101,
  "batch_no": "1",
  "case_name": "demo_case",
  "created_at": "2026-04-23T10:00:00",
  "row_names": ["R1", "R2"],
  "col_names": ["P1", "P2"],
  "points": [
    {
      "row_index": 0,
      "col_index": 0,
      "row_name": "R1",
      "col_name": "P1",
      "value": 0.12
    }
  ],
  "summary": {
    "response_count": 2,
    "parameter_count": 2,
    "point_count": 4
  }
}
```

POST /sensitivity/stored/matrix

接口含义：读取已存储的灵敏度矩阵。

功能说明：以“响应名 -> 参数值数组”的方式返回矩阵，适合算法调用。

影响数据库：

- `t_mt_py_fem_analysis_run`、`t_mt_py_fem_response_def`、`t_mt_py_fem_parameter_def`、`t_mt_py_fem_sensitivity_result`：读，读取已存储灵敏度矩阵并组装为矩阵载荷。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>batch_no</code>	<code>string</code>	否	批次号，默认 "1"

请求示例：

```
{
  "project_id": 101,
  "batch_no": "1"
}
```

返回 `data` 示例：

```
{
  "analysis_run_id": 12,
  "project_id": 101,
  "batch_no": "1",
  "case_name": "demo_case",
  "created_at": "2026-04-23T10:00:00",
  "row_names": ["R1", "R2"],
  "col_names": ["P1", "P2"],
  "rows": {
    "R1": [0.12, 0.22],
    "R2": [0.31, 0.41]
  },
  "summary": {
    "response_count": 2,
    "parameter_count": 2
  }
}
```

POST /sensitivity/stored/curve/parameter

接口含义：读取已存储的灵敏度（参数）曲线。

功能说明：按“每个响应一条曲线”的方式返回灵敏度数据，横轴为参数名，纵轴为该响应对各参数的灵敏度值，适合前端曲线图展示“响应-参数”关系。

影响数据库：

- `t_mt_py_fem_analysis_run`、`t_mt_py_fem_response_def`、`t_mt_py_fem_parameter_def`、`t_mt_py_fem_sensitivity_result`：读，读取已存储灵敏度矩阵并组装为参数曲线序列。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>batch_no</code>	<code>string</code>	否	批次号，默认 "1"

请求示例：

```
{
  "project_id": 101,
  "batch_no": "1"
}
```

返回 `data` 示例：

```
{
  "analysis_run_id": 12,
  "project_id": 101,
  "batch_no": "1",
  "case_name": "demo_case",
  "created_at": "2026-04-23T10:00:00",
  "curve_type": "parameter",
  "data": [
    {
      "label": "R1",
      "xaxis": ["P1", "P2"],
      "data": [["P1", 0.12], ["P2", 0.22]]
    },
    {
      "label": "R2",
      "xaxis": ["P1", "P2"],
      "data": [["P1", 0.31], ["P2", 0.41]]
    }
  ],
  "summary": {
    "response_count": 2,
    "parameter_count": 2,
    "curve_count": 2
  }
}
```

POST /sensitivity/stored/curve/response

接口含义：读取已存储的灵敏度（响应）曲线。

功能说明：按“每个参数一条曲线”的方式返回灵敏度数据，横轴为响应名，纵轴为该参数在各响应上的灵敏度值，适合前端曲线图展示“参数-响应”关系。

影响数据库：

- t_mt_py_fem_analysis_run、t_mt_py_fem_response_def、t_mt_py_fem_parameter_def、t_mt_py_fem_sensitivity_result：读，读取已存储灵敏度矩阵并转置组装为响应曲线序列。

请求参数：

参数名	类型	必填	说明
project_id	int	是	项目 ID
batch_no	string	否	批次号，默认 "1"

请求示例：

```
{
  "project_id": 101,
  "batch_no": "1"
}
```

返回 data 示例：

```
{
  "analysis_run_id": 12,
  "project_id": 101,
  "batch_no": "1",
  "case_name": "demo_case",
  "created_at": "2026-04-23T10:00:00",
  "curve_type": "response",
  "data": [
    {
      "label": "P1",
      "xaxis": ["R1", "R2"],
      "data": [["R1", 0.12], ["R2", 0.31]]
    },
    {
      "label": "P2",
      "xaxis": ["R1", "R2"],
      "data": [["R1", 0.22], ["R2", 0.41]]
    }
  ],
  "summary": {
    "response_count": 2,
    "parameter_count": 2,
    "curve_count": 2
  }
}
```

POST /sensitivity/workspace/build

接口含义：从 ODB 构建灵敏度 workspace。

功能说明：调用 `tools/run_t1.py` 将 ODB 转换为供后续查询使用的 workspace。

影响数据库：

- 无。该接口只生成 workspace 文件，不直接读写业务数据库。

请求参数：

参数名	类型	必填	说明
<code>odb_path</code>	<code>string</code>	是	ODB 文件路径
<code>workspace</code>	<code>string</code>	是	workspace 输出路径
<code>abaqus</code>	<code>string</code>	否	Abaqus 命令
<code>python3</code>	<code>string</code>	否	Python 解释器
<code>keep_raw</code>	<code>bool</code>	否	是否保留原始中间文件，默认 <code>false</code>

请求示例：

```
{
  "odb_path": "D:/demo/model.odb",
  "workspace": "D:/demo/workspace",
  "keep_raw": false
}
```

返回 `data` 示例：

```
{
  "odb_path": "D:/demo/model.odb",
  "workspace": "D:/demo/workspace",
  "manifest_db": "D:/demo/workspace/manifest.db",
  "stdout_tail": "...
}
```

POST /sensitivity/overview

接口含义：查看 workspace 总览。

功能说明：返回 workspace 中可用的步、帧、实例和结果文件目录。

影响数据库：

- 无。该接口只读取 workspace 内部清单，不直接读写业务数据库。

请求参数：

参数名	类型	必填	说明
<code>workspace</code>	<code>string</code>	是	workspace 路径

请求示例:

```
{
  "workspace": "D:/demo/workspace"
}
```

返回 data 示例:

```
{
  "workspace": "D:/demo/workspace",
  "steps": [
    {
      "step_name": "Step-1"
    }
  ],
  "frames": [
    {
      "step_name": "Step-1",
      "frame_idx": 0,
      "frame_value": 0.0,
      "description": "Initial"
    }
  ],
  "instances": [
    {
      "instance_name": "PART-1-1",
      "part_name": "PART-1",
      "node_count": 1200,
      "elem_count": 600
    }
  ],
  "result_files": [
    {
      "step_name": "Step-1",
      "field_name": "d_U_001",
      "file_path": "D:/demo/workspace/result.h5",
      "components": ["S"],
      "positions": ["CENTROID"]
    }
  ]
}
```

POST /sensitivity/table

接口含义: 按条件查询 workspace 中的灵敏度表格切片。

功能说明: 指定步、字段、实例、位置、分量和帧后, 返回表格列定义、行数据和矩阵。

影响数据库:

- 无。该接口只读取 workspace 结果文件, 不直接读写业务数据库。

请求参数:

参数名	类型	必填	说明
workspace	string	是	workspace 路径
step	string	否	步名；未传时自动解析唯一值
field	string	否	字段名；未传时自动解析唯一值
instance	string	否	实例名；未传时自动解析唯一值
position	string	否	位置，如 NODAL、CENTROID
components	string[]	否	分量列表
frame_indices	int[]	否	帧号列表
entity_labels	int[]	否	指定节点或单元标签
aggregation	string	否	聚合方式，默认 max_abs

请求示例：

```
{
  "workspace": "D:/demo/workspace",
  "step": "Step-1",
  "field": "d_u_001",
  "instance": "PART-1-1",
  "position": "CENTROID",
  "components": ["S"],
  "frame_indices": [0],
  "aggregation": "max_abs"
}
```

返回 data 示例：

```
{
  "workspace": "D:/demo/workspace",
  "step": "Step-1",
  "field": "d_u_001",
  "instance": "PART-1-1",
  "position": "CENTROID",
  "aggregation": "max_abs",
  "components": ["S"],
  "entity_labels": null,
  "columns": [
    {
      "key": "S",
      "title": "S",
      "type": "number",
      "component": "S"
    }
  ],
  "rows": [
    {
      "row_id": "Step-1:0",
```



```

        "frame_idx": 0,
        "frame_value": 0.0,
        "s": 0.12
    }
],
"matrix": [[0.12]],
"row_meta": [
    {
        "step": "Step-1",
        "frame_idx": 0,
        "frame_value": 0.0,
        "description": "Initial"
    }
],
"summary": {
    "row_count": 1,
    "column_count": 1,
    "value_abs_max": 0.12,
    "missing_labels": []
}
}

```

POST /sensitivity/export/vtu

接口含义：导出 ODB/DSA 灵敏度结果到 VTU。

功能说明：通用导出接口，当前实际走 DSA 导出逻辑。

影响数据库：

- `t_mt_py_fem_selected_parameter`：读，读取优化参数与单元映射，用于给导出字段命名和回填参数信息。
- `t_mt_py_fem_node_octree_cache`：读，在未显式传入 `inp_path` 时推断项目关联 INP 路径。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>output_vtu</code>	<code>string</code>	是	输出 VTU 路径
<code>odb_id</code>	<code>string</code>	否	已加载 ODB 标识
<code>base_url</code>	<code>string</code>	否	L3 服务地址
<code>inp_path</code>	<code>string</code>	否	INP 路径；不传时尝试从项目推断
<code>workspace</code>	<code>string</code>	否	workspace 路径
<code>odb_path</code>	<code>string</code>	否	ODB 路径
<code>step</code>	<code>string</code>	否	结果步名
<code>instances</code>	<code>string[]</code>	否	实例过滤列表

参数名	类型	必填	说明
field_prefix	string	否	字段前缀，默认 d_u_
response_component	string	否	响应分量
position	string	否	结果位置
aggregation	string	否	聚合方式，默认 max_abs
frame	int	否	帧号，默认 0
abaqus	string	否	Abaqus 命令
python3	string	否	Python 解释器
keep_raw	bool	否	是否保留中间文件，默认 false
timeout	int	否	超时秒数，默认 60

请求示例：

```
{
  "project_id": 101,
  "output_vtu": "D:/demo/sensitivity.vtu",
  "workspace": "D:/demo/workspace",
  "inp_path": "D:/demo/model.inp",
  "step": "Step-1",
  "instances": ["PART-1-1"]
}
```

返回 data 示例：

```
{
  "project_id": 101,
  "odb_id": "demo-odb-id",
  "base_url": "http://127.0.0.1:18765",
  "workspace": "D:/demo/workspace",
  "source_mode": "workspace",
  "workspace_built": false,
  "inp_path": "D:/demo/model.inp",
  "output_vtu": "D:/demo/sensitivity.vtu",
  "step": "Step-1",
  "instances": ["PART-1-1"],
  "frame": 0,
  "aggregation": "max_abs",
  "exported_field_count": 3,
  "exported_fields": ["d_u_001", "d_u_002", "d_u_003"],
  "items": []
}
```

POST /sensitivity/export/vtu/dsa

接口含义：显式导出 DSA 灵敏度结果到 VTU。

功能说明：与 /sensitivity/export/vtu 类似，但语义上明确为 DSA 导出。

影响数据库：

- `t_mt_py_fem_selected_parameter`：读，读取优化参数与单元映射。
- `t_mt_py_fem_node_octree_cache`：读，在未显式传入 `inp_path` 时推断项目关联 INP 路径。

请求参数：与 /sensitivity/export/vtu 一致。

请求示例：

```
{
  "project_id": 101,
  "output_vtu": "D:/demo/sensitivity_dsa.vtu",
  "workspace": "D:/demo/workspace",
  "inp_path": "D:/demo/model.inp",
  "step": "Step-1",
  "instances": ["PART-1-1"]
}
```

返回 data 示例：与 /sensitivity/export/vtu 一致。

POST /sensitivity/export/vtu/adjoint

接口含义：导出伴随灵敏度结果到 VTU。

功能说明：与 DSA 导出类似，但通过明确的 `field_name` 指定伴随灵敏度字段。

影响数据库：

- `t_mt_py_fem_node_octree_cache`：读，在未显式传入 `inp_path` 时推断项目关联 INP 路径。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>output_vtu</code>	<code>string</code>	是	输出 VTU 路径
<code>field_name</code>	<code>string</code>	是	伴随灵敏度字段名
<code>odb_id</code>	<code>string</code>	否	已加载 ODB 标识
<code>base_url</code>	<code>string</code>	否	L3 服务地址
<code>inp_path</code>	<code>string</code>	否	INP 路径
<code>workspace</code>	<code>string</code>	否	workspace 路径
<code>odb_path</code>	<code>string</code>	否	ODB 路径
<code>step</code>	<code>string</code>	否	结果步名

参数名	类型	必填	说明
instances	string[]	否	实例过滤列表
position	string	否	结果位置
aggregation	string	否	聚合方式，默认 max_abs
frame	int	否	帧号，默认 0
abaqus	string	否	Abaqus 命令
python3	string	否	Python 解释器
keep_raw	bool	否	是否保留中间文件，默认 false
timeout	int	否	超时秒数，默认 60

请求示例：

```
{
  "project_id": 101,
  "output_vtu": "D:/demo/adjoint.vtu",
  "field_name": "ADJOINT_FIELD",
  "workspace": "D:/demo/workspace",
  "inp_path": "D:/demo/model.inp",
  "step": "Step-1",
  "instances": ["PART-1-1"]
}
```

返回 data 示例：

```
{
  "project_id": 101,
  "output_vtu": "D:/demo/adjoint.vtu",
  "field_name": "ADJOINT_FIELD",
  "step": "Step-1",
  "instances": ["PART-1-1"],
  "items": []
}
```

8. 求解器接口

POST /solver/abaqus/sensitivity

接口含义：生成并可执行 Abaqus 灵敏度分析工况。

功能说明：先生成灵敏度分析 INP，再根据 run_solver 决定是否立即调用 Abaqus。

影响数据库：

- 无。该接口只生成输入文件并可调用求解器，不直接读写业务数据库。

请求参数：

参数名	类型	必填	说明
input_inp	string	是	原始 INP 路径
output_dir	string	否	输出目录
response_elset	string	否	响应 ELSET
response_nset	string	否	响应 NSET
response_frequency	int	否	输出频率，默认 1
node_vars	string[]	否	节点变量列表
element_vars	string[]	否	单元变量列表
abaqus	string	否	Abaqus 命令
job_name	string	否	作业名
cpus	int	否	CPU 数
interactive	bool	否	是否 interactive 运行，默认 true
run_solver	bool	否	是否立即求解，默认 true
timeout_sec	int	否	求解超时秒数
extra_args	string[]	否	额外命令参数

请求示例：

```
{
  "input_inp": "D:/demo/model.inp",
  "output_dir": "D:/demo/out",
  "response_nset": "NSET_RESP",
  "node_vars": ["U"],
  "element_vars": ["S"],
  "run_solver": false
}
```

返回 data 示例：

```
{
  "workflow": "abaqus_sensitivity",
  "input_inp": "D:/demo/model.inp",
  "output_dir": "D:/demo/out",
  "job_name": "model_sensitivity",
  "generated_files": {
    "design_parameter_inp": "D:/demo/out/design_parameters.inp",
    "analysis_inp": "D:/demo/out/model_sensitivity.inp"
  },
  "command_preview": [
    "abaqus",
    "job=model_sensitivity",
    "input=model_sensitivity.inp",
  ]
}
```

```
    "interactive"
  ],
  "solver": {
    "ok": true,
    "returncode": 0,
    "command": [],
    "workdir": "D:/demo/out",
    "stdout_tail": "",
    "stderr_tail": "",
    "artifacts": {
      "odb": "D:/demo/out/model_sensitivity.odb"
    }
  }
}
```

POST /solver/abaqus/adjoint

接口含义：生成并可执行 Abaqus 伴随分析工况。

功能说明：将原始 INP 改写为伴随壳厚分析工况，再根据 `run_solver` 决定是否执行 Abaqus。

影响数据库：

- 无。该接口只改写输入文件并可调用求解器，不直接读写业务数据库。

请求参数：

参数名	类型	必填	说明
<code>input_inp</code>	<code>string</code>	是	原始 INP 路径
<code>output_inp</code>	<code>string</code>	否	输出 INP 路径；不传则自动生成
<code>response_nset</code>	<code>string</code>	否	响应 NSET
<code>abaqus</code>	<code>string</code>	否	Abaqus 命令
<code>job_name</code>	<code>string</code>	否	作业名
<code>cpus</code>	<code>int</code>	否	CPU 数
<code>interactive</code>	<code>bool</code>	否	是否 interactive 运行，默认 <code>true</code>
<code>run_solver</code>	<code>bool</code>	否	是否立即求解，默认 <code>true</code>
<code>timeout_sec</code>	<code>int</code>	否	求解超时秒数
<code>extra_args</code>	<code>string[]</code>	否	额外命令参数

请求示例：

```
{
  "input_inp": "D:/demo/model.inp",
  "output_inp": "D:/demo/model_adjoint_thickness.inp",
  "response_nset": "NSET_RESP",
  "run_solver": false
}
```

返回 data 示例:

```
{
  "workflow": "abaqus_adjoint_shell",
  "input_inp": "D:/demo/model.inp",
  "output_inp": "D:/demo/model_adjoint_thickness.inp",
  "job_name": "model_adjoint_thickness",
  "generated_files": {
    "analysis_inp": "D:/demo/model_adjoint_thickness.inp"
  },
  "command_preview": [
    "abaqus",
    "job=model_adjoint_thickness",
    "input=model_adjoint_thickness.inp",
    "interactive"
  ],
  "solver": null
}
```

POST /solver/nastran/sol103

接口含义: 生成并可执行 Nastran SOL103 工况。

功能说明: 将输入 BDF 转换为 SOL103 分析卡片, 再根据 `run_solver` 决定是否调用 Nastran。

影响数据库:

- 无。该接口只改写输入文件并可调用求解器, 不直接读写业务数据库。

请求参数:

参数名	类型	必填	说明
<code>input_bdf</code>	<code>string</code>	是	原始 BDF 路径
<code>output_bdf</code>	<code>string</code>	否	输出 BDF 路径; 不传则自动生成
<code>settings</code>	<code>object</code>	否	SOL103 转换设置; 支持 <code>embed_geometry</code> 开关
<code>nastran</code>	<code>string</code>	否	Nastran 可执行命令, 默认 <code>nastran</code>
<code>run_solver</code>	<code>bool</code>	否	是否立即求解, 默认 <code>true</code>
<code>timeout_sec</code>	<code>int</code>	否	求解超时秒数
<code>extra_args</code>	<code>string[]</code>	否	额外命令参数

请求示例:

```
{
  "input_bdf": "D:/demo/model.bdf",
  "output_bdf": "D:/demo/model_sol103.bdf",
  "settings": {
    "method_id": 1
  },
  "run_solver": false
}
```

返回 data 示例:

```
{
  "workflow": "nastran_sol103",
  "input_bdf": "D:/demo/model.bdf",
  "output_bdf": "D:/demo/model_sol103.bdf",
  "command_preview": [
    "nastran",
    "D:/demo/model_sol103.bdf"
  ],
  "solver": null
}
```

9. DOF 匹配、响应目录、结果导入与相关性接口

POST /match/dofs

接口含义：试验自由度与 FEM 自由度匹配。

功能说明：基于已完成的节点匹配关系，为每个试验点生成 UX/UY/UZ 到 FEM 自由度的映射，并写入 `t_mt_py_fem_dof_match`。在 `modal_unv` 模式下，会先根据试验模态数据计算每个平动方向的 `match_score`，低于阈值的 DOF 不写入匹配表。

影响数据库：

- `t_mt_py_test_modal_shape_real`、`t_mt_py_test_modal_shape_imag`：读，分析试验模态方向信息。
- `t_mt_py_fem_node_match`：读，读取节点匹配结果。
- `t_mt_py_fem_dof_match`：写，保存试验 DOF 到 FEM DOF 的映射。
- `t_mt_py_fem_response_catalog`、`t_mt_py_fem_modal_correlation`：写，`overwrite=true` 时清理依赖旧 DOF 匹配的下游结果。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>overwrite</code>	<code>bool</code>	否	是否覆盖旧结果，默认 <code>true</code>
<code>min_match_score</code>	<code>float</code>	否	DOF 匹配分数阈值；仅 <code>modal_unv</code> 模式使用，不传时默认 <code>1e-8</code>

请求示例:

```
{
  "project_id": 101,
  "overwrite": true,
  "min_match_score": 0.05
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "node_match_count": 30,
  "dof_match_count": 74,
  "rejected_dof_count": 16,
  "min_match_score": 0.05,
  "dof_matches_preview": [
    {
      "test_node_id": "1",
      "test_dof": "UX",
      "instance_name": "PART-1-1",
      "part_name": "PART-1",
      "fem_node_label": 1001,
      "fem_dof": "U1",
      "direction": [1.0, 0.0, 0.0],
      "match_score": 1.0,
      "transform": {}
    }
  ],
  "rejected_dof_preview": [
    {
      "test_node_id": "8",
      "test_dof": "UZ",
      "match_score": 0.012
    }
  ]
}
```

补充说明 (2026-05-10) :

- `modal_unv` 模式下, `min_match_score` 的实际含义已经调整为“试验测量值阈值”。
- 程序会读取已导入的试验模态振型, 对每个测点的 `ux/uy/uz` 三个方向分别统计“全部已导入模态中的最大绝对测量值”。
- 如果某个方向的最大绝对值小于 `min_match_score`, 则认为该方向数据不可用, 不写入 `t_mt_py_fem_dof_match`。
- 后续 `MAC` 计算只会使用已经通过该阈值筛选的 DOF 匹配对。
- `dof_matches_preview[*].match_score` 与 `rejected_dof_preview[*].match_score` 现在表示该方向的“最大绝对测量值”, 不再表示三方向相对占比。

POST /match/dofs/query

接口含义：查询已保存的自由度匹配关系。

功能说明：读取 `t_mt_py_fem_dof_match` 中的全部映射。当前对外同时保留两个查询入口：

- `POST /match/dofs/query`： `app.py` 中的简化入口。
- `POST /get/dof_match_result`： `webapi/routers/matching.py` 中的兼容入口。

影响数据库：

- `t_mt_py_fem_dof_match`：读，返回当前项目全部 DOF 匹配记录。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 data 示例：

```
{
  "project_id": 101,
  "dof_matches": [
    {
      "test_node_id": "1",
      "test_dof": "UX",
      "instance_name": "PART-1-1",
      "part_name": "PART-1",
      "fem_node_label": 1001,
      "fem_dof": "U1",
      "direction_x": 1.0,
      "direction_y": 0.0,
      "direction_z": 0.0,
      "match_score": 1.0,
      "transform_json": "{}"
    }
  ]
}
```

POST /catalog/response/build

接口含义：构建响应目录。

功能说明：把试验模态频率和已匹配 DOF 统一整理为响应目录，供灵敏度、模型修正和相关性流程使用。

影响数据库：

- `t_mt_py_test_modal_frequency`：读，生成模态频率响应项。
- `t_mt_py_fem_dof_match`：读，生成节点自由度响应项。
- `t_mt_py_fem_response_catalog`：写，保存统一响应目录。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>overwrite</code>	<code>bool</code>	否	是否覆盖旧目录，默认 <code>true</code>
<code>include_test_modes</code>	<code>bool</code>	否	是否把试验模态频率写入目录，默认 <code>true</code>
<code>include_node_dofs</code>	<code>bool</code>	否	是否把节点 DOF 响应写入目录，默认 <code>true</code>

请求示例：

```
{
  "project_id": 101,
  "overwrite": true,
  "include_test_modes": true,
  "include_node_dofs": true
}
```

返回 `data` 示例：

```
{
  "project_id": 101,
  "response_count": 12,
  "modal_frequency_count": 6,
  "nodal_response_count": 6,
  "responses_preview": [
    {
      "response_code": "MODE_FREQ:1",
      "response_name": "Mode 1 frequency",
      "response_type": "MODAL_FREQUENCY",
      "entity_type": "MODE",
      "component": "FREQ"
    }
  ]
}
```

POST /catalog/response

接口含义：查询响应目录。

功能说明：读取 `t_mt_py_fem_response_catalog` 的完整内容。

影响数据库：

- `t_mt_py_fem_response_catalog`：读，返回当前项目已构建的响应目录。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例:

```
{
  "project_id": 101
}
```

返回 `data` 示例:

```
{
  "project_id": 101,
  "responses": [
    {
      "response_code": "NODE_DOF:PART-1-1:1001:U1:1:UX",
      "response_name": "PART-1-1:1001 U1",
      "response_type": "NODAL_DISPLACEMENT",
      "entity_type": "NODE_DOF",
      "component": "U1"
    }
  ]
}
```

POST `/import/fem/modal`

接口含义: 导入 FEM 模态结果。

功能说明: 支持从 JSON 文件或请求体数组导入 FEM 模态结果, 写入逐节点模态表。

补充说明: `file_path` 与 `modes` 至少传一个。

影响数据库:

- `t_mt_py_fem_modal_result`: 写, 保存 FEM 模态结果。
- `t_mt_py_fem_modal_correlation`: 写, `overwrite=true` 时清理依赖旧 FEM 模态结果的模态相关性。

请求参数:

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>overwrite</code>	<code>bool</code>	否	是否覆盖旧 FEM 模态结果, 默认 <code>true</code>
<code>file_path</code>	<code>string</code>	否	模态结果 JSON 路径
<code>modes</code>	<code>object[]</code>	否	模态结果数组

请求示例:

```
{
  "project_id": 101,
  "overwrite": true,
  "file_path": "D:/demo/fem_modal.json"
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "mode_count": 6,
  "row_count": 7200,
  "source_file_path": "D:/demo/fem_modal.json",
  "rows_preview": [
    {
      "mode_no": 1,
      "frequency": 32.5,
      "instance_name": "PART-1-1",
      "part_name": "PART-1",
      "fem_node_label": 1001,
      "u1": 0.01,
      "u2": 0.0,
      "u3": 0.0,
      "extra_json": {}
    }
  ]
}
```

POST /import/fem/modal/query

接口含义: 查询 FEM 模态结果。

功能说明: 读取 `t_mt_py_fem_modal_result` 中的全部模态节点结果。

影响数据库:

- `t_mt_py_fem_modal_result`: 读, 返回已导入 FEM 模态结果。

请求参数:

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例:

```
{
  "project_id": 101
}
```

返回 data 示例:

```
{
  "project_id": 101,
```

```
"rows": [
  {
    "mode_no": 1,
    "frequency": 32.5,
    "instance_name": "PART-1-1",
    "part_name": "PART-1",
    "fem_node_label": 1001,
    "u1": 0.01,
    "u2": 0.0,
    "u3": 0.0
  }
]
```

POST /import/fem/static

接口含义：导入 FEM 静力结果。

功能说明：支持从文本文件或请求体数组导入静力位移/转角结果，写入 `t_mt_py_fem_static_result`。

补充说明：`file_path` 与 `rows` 至少传一个。

影响数据库：

- `t_mt_py_fem_static_result`：写，保存 FEM 静力结果。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>overwrite</code>	<code>bool</code>	否	是否覆盖旧静力结果，默认 <code>true</code>
<code>file_path</code>	<code>string</code>	否	静力结果文本路径
<code>rows</code>	<code>object[]</code>	否	静力结果数组
<code>load_case_no</code>	<code>int</code>	否	默认工况号，默认 1
<code>instance_name</code>	<code>string</code>	否	默认实例名
<code>part_name</code>	<code>string</code>	否	默认零件名

请求示例：

```
{
  "project_id": 101,
  "overwrite": true,
  "file_path": "D:/demo/static.txt",
  "load_case_no": 1,
  "instance_name": "PART-1-1",
  "part_name": "PART-1"
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "load_case_nos": [1],
  "row_count": 1200,
  "source_file_path": "D:/demo/static.txt",
  "rows_preview": [
    {
      "load_case_no": 1,
      "instance_name": "PART-1-1",
      "part_name": "PART-1",
      "fem_node_label": 1001,
      "u1": 0.1,
      "u2": 0.0,
      "u3": 0.0,
      "ur1": 0.0,
      "ur2": 0.0,
      "ur3": 0.0
    }
  ]
}
```

POST /import/fem/static/query

接口含义: 查询 FEM 静力结果。

功能说明: 按项目和可选工况号读取 FEM 静力结果。

影响数据库:

- t_mt_py_fem_static_result: 读, 返回指定项目或工况的 FEM 静力结果。

请求参数:

参数名	类型	必填	说明
project_id	int	是	项目 ID
load_case_no	int	否	工况号; 不传则返回全部工况

请求示例:

```
{
  "project_id": 101,
  "load_case_no": 1
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "load_case_no": 1,
  "rows": [
    {
```

```
    "load_case_no": 1,
    "instance_name": "PART-1-1",
    "part_name": "PART-1",
    "fem_node_label": 1001,
    "u1": 0.1,
    "u2": 0.0,
    "u3": 0.0,
    "ur1": 0.0,
    "ur2": 0.0,
    "ur3": 0.0
  }
]
}
```

POST /correlation/modal/compute

接口含义：计算模态相关性。

功能说明：基于试验模态、FEM 模态和 DOF 匹配关系，枚举试验模态与 FEM 模态组合，计算 DAC/DSF/MAC，并落库到模态相关性表。MAC 只使用已通过 DOF 匹配阈值筛选的那些自由度对。

影响数据库：

- `t_mt_py_test_modal_shape_real`、`t_mt_py_test_modal_shape_imag`、`t_mt_py_test_modal_shape`、`t_mt_py_test_modal_frequency`：读，加载试验模态向量和频率。
- `t_mt_py_fem_modal_result`：读，加载 FEM 模态结果。
- `t_mt_py_fem_dof_match`：读，建立模态比较的自由度对应关系。
- `t_mt_py_fem_modal_correlation`：写，保存模态相关性结果。
- `t_mt_py_fem_static_shape_pairs`：写，保存参与比较的静形配对明细。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>overwrite</code>	<code>bool</code>	否	是否覆盖旧相关性结果，默认 <code>true</code>

请求示例：

```
{
  "project_id": 101,
  "overwrite": true
}
```

返回 `data` 示例：

```
{
  "project_id": 101,
  "comparison_count": 36,
  "best_pairs_by_test_mode": [
    {
```



```
    "test_mode_no": 1,
    "fem_mode_no": 2,
    "dof_pair_count": 74,
    "dac": 97.3,
    "mac": 0.973,
    "dsf": 1.01,
    "freq_test": 32.5,
    "freq_fem": 31.8,
    "freq_error_ratio": -0.0215
  }
],
"results_preview": []
}
```

POST /correlation/modal

接口含义：查询模态相关性结果。

功能说明：读取 `t_mt_py_fem_modal_correlation` 中已保存的 `MAC` 结果，并返回一个矩阵。矩阵的行是 FEM 模态阶次，列是试验模态阶次。

影响数据库：

- `t_mt_py_fem_modal_correlation`：读，返回当前项目已保存的模态相关性矩阵。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 101
}
```

返回 `data` 示例：

```
{
  "project_id": 101,
  "row_mode_order": [1, 2, 3, 4],
  "column_mode_order": [1, 2, 3],
  "matrix": [
    [0.12, 0.973, 0.08],
    [0.05, 0.11, 0.962],
    [0.02, 0.07, 0.15],
    [0.01, 0.03, 0.09]
  ]
}
```

11. Nastran 模态闭环补充接口

POST /solver/nastran/sol103/preview

接口含义：预览 Nastran SOL103 输入卡片。

功能说明：读取原始 BDF 和设置项，生成 SOL103 控制段预览，不真正写文件也不调用求解器。

影响数据库：

- 无。该接口只做输入预览，不读写业务数据库。

请求参数：

参数名	类型	必填	说明
input_bdf	string	是	原始 BDF 路径
settings	object	否	SOL103 转换设置

请求示例：

```
{
  "input_bdf": "D:/demo/model.bdf",
  "settings": {
    "dynamic.fmin": 0.0,
    "dynamic.fmax": 1000.0,
    "dynamic.vectors": 20,
    "dynamic.norm": 2,
    "result.target": "OP2",
    "embed_geometry": true
  }
}
```

settings 补充说明：

- result.target：控制结果输出目标，支持 OP2、F06、BOTH
- embed_geometry：是否希望生成的 OP2 带嵌入几何模型
- 当 result.target 为 OP2 或 BOTH 时：
 - embed_geometry=true：生成 PARAM POST -1
 - embed_geometry=false：生成 PARAM POST -2
- 当 result.target=F06 时，不生成 OP2，此时 embed_geometry 不起作用
- 如果显式传入 settings.post，则仍以 post 为最高优先级，覆盖 embed_geometry

返回 data 示例：

```
{
  "workflow": "nastran_sol103_preview",
  "input_bdf": "D:/demo/model.bdf",
  "control_lines_preview": [
    "SOL 103",
    "CEND",
    "METHOD = 1",
  ]
}
```

```
"BEGIN BULK"
],
"filtered_cards": {
  "removed_eig_cards": 1,
  "removed_param_cards": 2
},
"warnings": []
}
```

POST /solver/nastran/sol103/generate

接口含义：生成 SOL103 BDF 文件。

功能说明：基于原始 BDF 和设置项输出新的 `sol103.bdf`，但不调用求解器。

影响数据库：

- 无。该接口只生成求解输入文件，不读写业务数据库。

请求参数：

参数名	类型	必填	说明
<code>input_bdf</code>	<code>string</code>	是	原始 BDF 路径
<code>output_bdf</code>	<code>string</code>	否	输出 BDF 路径；不传则自动生成
<code>settings</code>	<code>object</code>	否	SOL103 转换设置

请求示例：

```
{
  "input_bdf": "D:/demo/model.bdf",
  "output_bdf": "D:/demo/model_sol103.bdf",
  "settings": {
    "dynamic.fmin": 0.0,
    "dynamic.fmax": 1000.0,
    "dynamic.vectors": 20,
    "dynamic.norm": 2,
    "result.target": "OP2",
    "embed_geometry": true
  }
}
```

返回 `data` 示例：

```
{
  "workflow": "nastran_sol103_generate",
  "input_bdf": "D:/demo/model.bdf",
  "output_bdf": "D:/demo/model_sol103.bdf",
  "generated_files": {
    "analysis_bdf": "D:/demo/model_sol103.bdf"
  },
  "warnings": []
}
```

POST /solver/nastran/sol103/run

接口含义：生成并执行 SOL103 求解。

功能说明：先生成 SOL103 BDF，再根据配置或请求参数调用 Nastran，返回求解产物路径和产物摘要。

影响数据库：

- 无。该接口只生成/求解文件，不直接写业务数据库。

请求参数：

参数名	类型	必填	说明
input_bdf	string	是	原始 BDF 路径
output_bdf	string	否	输出 BDF 路径；不传则自动生成
settings	object	否	SOL103 转换设置
nastran	string	否	Nastran 可执行命令；不传则优先读 service_config.json 中的 NASTRAN
run_solver	bool	否	是否立即求解，默认 true
timeout_sec	int	否	求解超时秒数
extra_args	string[]	否	额外命令参数
async_submit	bool	否	是否异步提交，默认 false

请求示例：

```
{
  "input_bdf": "D:/demo/model.bdf",
  "output_bdf": "D:/demo/model_sol103.bdf",
  "settings": {
    "dynamic.fmin": 0.0,
    "dynamic.fmax": 1000.0,
    "dynamic.vectors": 20,
    "dynamic.norm": 2,
    "result.target": "OP2",
    "embed_geometry": true
  },
  "run_solver": true,
  "timeout_sec": 600,
  "extra_args": [],
  "async_submit": false
}
```

返回 data 示例：

```
{
  "workflow": "nastran_sol103_run",
}
```

```
"input_bdf": "D:/demo/model.bdf",
"output_bdf": "D:/demo/model_sol103.bdf",
"command_preview": [
  "nastran",
  "D:/demo/model_sol103.bdf"
],
"solver": {
  "ok": true,
  "returncode": 0,
  "artifacts": {
    "bdf": "D:/demo/model_sol103.bdf",
    "f06": "D:/demo/model_sol103.f06",
    "op2": "D:/demo/model_sol103.op2"
  }
},
"artifacts_summary": {
  "has_f06": true,
  "has_op2": true
},
"warnings": []
}
```

GET /solver/nastran/tasks?task_id=...

接口含义：查询 Nastran 后台任务状态。

功能说明：用于查询异步提交的 SOL103/SOL200 求解、OP2 导入等后台任务状态。

影响数据库：

- 无。该接口只读取内存中的后台任务状态，不读写业务数据库。

请求参数：

参数名	类型	必填	说明
task_id	string	是	后台任务 ID

请求示例：

```
GET /solver/nastran/tasks?task_id=2b7f7b72-xxxx
```

返回 data 示例：

```
{
  "task_id": "2b7f7b72-xxxx",
  "task_type": "solver.nastran.sol103.run",
  "status": "succeeded",
  "request": {},
  "progress": null,
  "result": {
    "workflow": "nastran_sol103_run"
  },
  "error": null
}
```

POST /import/op2/model/preview

接口含义：预览 OP2 模态结果。

功能说明：读取 OP2 中的模态结果块，可选结合 BDF 按节点号对齐，只返回摘要和少量节点预览，不入库。

影响数据库：

- 无。该接口只读取 OP2/BDF 文件，不读写业务数据库。

请求参数：

参数名	类型	必填	说明
op2_path	string	是	OP2 文件路径
project_id	int	否	传入后会在预览完成后自动将模态结果写入数据库
bdf_path	string	否	BDF 文件路径；传入后会按节点号对齐
subcase_id	int	否	模态子工况号
mode_numbers	int[]	否	指定导入或预览的模态阶次
preview_node_limit	int	否	每阶最多返回多少个节点预览，默认 5
overwrite	bool	否	当 project_id 存在并自动入库时是否覆盖旧 FEM 模态结果，默认 true
instance_name	string	否	自动入库时手工指定实例名
part_name	string	否	自动入库时手工指定零件名
async_submit	bool	否	当 project_id 存在并自动入库时是否异步提交，默认 false

请求示例：

```
{
  "project_id": 101,
  "op2_path": "D:/demo/model_sol103.op2",
  "bdf_path": "D:/demo/model.bdf",
  "preview_node_limit": 5,
  "overwrite": true,
  "async_submit": false
}
```

返回 data 示例：

```
{
  "workflow": "op2_model_preview",
  "source": {
    "op2_path": "D:/demo/model_sol103.op2",
```

```

    "bdf_path": "D:/demo/model.bdf",
    "mode": "bdf_plus_op2"
  },
  "subcases": [
    {
      "subcase_id": 1,
      "modes": [
        {
          "mode_no": 1,
          "frequency": 32.5,
          "eigenvalue": 41706.0,
          "node_count": 1200,
          "nodes_preview": [
            {
              "node_id": 1001,
              "u1": 0.01,
              "u2": 0.0,
              "u3": 0.0,
              "ur1": 0.0,
              "ur2": 0.0,
              "ur3": 0.0
            }
          ]
        }
      ]
    }
  ],
  "warnings": [],
  "store": {
    "project_id": 101,
    "mode_count": 24,
    "row_count": 28800
  }
}

```

补充说明：

- 当请求中不传 `project_id` 时，该接口仅预览，不写数据库。
- 当请求中传入 `project_id` 时，该接口会在预览完成后自动执行一次与 `POST /import/op2/modal/store` 等价的入库流程。
- 自动入库默认行为为：全部阶次导入、覆盖旧结果、同步执行；对应默认值分别是 `mode_numbers=null`、`overwrite=true`、`async_submit=false`。

POST /import/op2/modal/store

接口含义：将 OP2 模态结果导入数据库。

功能说明：读取 OP2 模态结果，整理成系统现有 FEM 模态表结构后写入数据库。对 BDF 路径导入时，默认按 `BDF_MODEL` 作为实例名落库，保证后续和 BDF 节点匹配结果一致。

影响数据库：

- `t_mt_py_fem_modal_result`：写，保存逐节点 FEM 模态结果。

- `t_mt_py_fem_modal_correlation`: 写, `overwrite=true` 时清理依赖旧 FEM 模态结果的模态相关性。

请求参数:

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>op2_path</code>	<code>string</code>	是	OP2 文件路径
<code>bdf_path</code>	<code>string</code>	否	BDF 文件路径; 传入后会按节点号对齐
<code>subcase_id</code>	<code>int</code>	否	模态子工况号
<code>mode_numbers</code>	<code>int[]</code>	否	指定导入的模态阶次
<code>overwrite</code>	<code>bool</code>	否	是否覆盖旧 FEM 模态结果, 默认 <code>true</code>
<code>instance_name</code>	<code>string</code>	否	手工指定实例名
<code>part_name</code>	<code>string</code>	否	手工指定零件名
<code>async_submit</code>	<code>bool</code>	否	是否异步提交, 默认 <code>false</code>

请求示例:

```
{
  "project_id": 101,
  "op2_path": "D:/demo/model_sol103.op2",
  "bdf_path": "D:/demo/model.bdf",
  "subcase_id": 1,
  "mode_numbers": [1, 2, 3, 4],
  "overwrite": true,
  "async_submit": false
}
```

返回 data 示例:

```
{
  "project_id": 101,
  "mode_count": 4,
  "row_count": 4800,
  "source_file_path": null,
  "rows_preview": [
    {
      "mode_no": 1,
      "frequency": 32.5,
      "instance_name": "BDF_MODEL",
      "part_name": "BDF_MODEL",
      "fem_node_label": 1001,
      "u1": 0.01,
      "u2": 0.0,
      "u3": 0.0,
      "extra_json": {
```



```
        "node_id": 1001,
        "subcase_id": 1
    }
}
],
"warnings": []
}
```

POST /export/op2/modal/vtu

接口含义：将指定阶 OP2 模态结果导出为 VTU。

功能说明：读取 OP2 与 BDF，按指定模态阶次组装网格和模态位移场，输出可用于 ParaView 等工具查看的 VTU 文件。

影响数据库：

- 无。该接口只读取 OP2/BDF 文件并写出 VTU 文件，不读写业务数据库。

请求参数：

参数名	类型	必填	说明
op2_path	string	是	OP2 文件路径
bdf_path	string	是	BDF 文件路径
output_vtu	string	是	输出 VTU 路径
mode_number	int	是	指定模态阶次
subcase_id	int	否	模态子工况号
displacement_scale	float	否	变形缩放系数，默认 1.0

请求示例：

```
{
  "op2_path": "D:/demo/model_sol103.op2",
  "bdf_path": "D:/demo/model.bdf",
  "output_vtu": "D:/demo/model_1.vtu",
  "mode_number": 1,
  "subcase_id": 1,
  "displacement_scale": 10.0
}
```

返回 data 示例：

```
{
  "workflow": "op2_modal_vtu_export",
  "output_vtu": "D:/demo/mode_1.vtu",
  "mode_no": 1,
  "frequency": 32.5,
  "point_count": 1200,
  "cell_block_count": 3,
  "warnings": []
}
```

POST /correlation/static/compute

接口含义：计算静力相关性但不落库。

功能说明：计算试验静力结果与 FEM 静力结果之间的 DAC、DSF 和误差预览，适合调试或预览；与 `/correlation/evaluate` 的区别是本接口不写数据库。

影响数据库：

- `t_mt_py_test_static_result`、`t_mt_py_fem_static_result`、`t_mt_py_fem_node_match`：读，读取试验结果、FEM 结果和节点映射。
- 无写入。该接口只返回预览结果，不落库。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>load_case_no</code>	<code>int</code>	否	工况号
<code>result_no</code>	<code>int</code>	否	试验结果号
<code>components</code>	<code>string[]</code>	否	比较分量列表
<code>include_rotations</code>	<code>bool</code>	否	是否包含转角分量，默认 <code>false</code>

请求示例：

```
{
  "project_id": 101,
  "load_case_no": 1,
  "result_no": 1,
  "components": ["UX", "UY", "UZ"],
  "include_rotations": false
}
```

返回 `data` 示例：

```
{
  "project_id": 101,
  "load_case_no": 1,
  "result_no": 1,
  "components": ["UX", "UY", "UZ"],
```

```
"aligned_point_count": 30,
"value_count": 90,
"component_value_counts": {
  "ux": 30,
  "uy": 30,
  "uz": 30
},
"analysis_error_row_count": 90,
"analysis_error_preview": [],
"dac": 98.5,
"dsf": 1.02,
"extra": {
  "anchors_preview": []
}
}
```

Nastran SOL200 / 灵敏度接口补充说明 (2026-05-11)

- `POST /solver/nastran/sol200/preview`
- `POST /solver/nastran/sol200/generate`
- `POST /solver/nastran/sol200/run`

补充说明:

- SOL200 现在默认按 BAS 逻辑使用质量归一化特征向量, 也就是 `dynamic.norm=MASS`。
- 如果显式传入的 `dynamic.norm` 不是 `MASS/2`, 接口会直接报错, 避免生成一份能运行但灵敏度语义不对的 deck。
- SOL200 现在默认使用更贴近 BAS 的 `PARAM POST=-5`, 优先为灵敏度后处理保留结果文件。
- SOL200 现在支持 `include` 模式生成:
 - 主 deck 继续走 `output_bdf`
 - 设计卡会单独写到 `design_model.bdf`
 - 请求方式: `settings.sol200.deck_mode = "include"`
- SOL200 现在支持参数便捷预设: `parameter_preset.preset = "all_used_material_e_rho"`
 - 服务会扫描 `input_bdf` 中所有被单元实际用到的 `MAT1`
 - 自动为每个材料生成 `E` 和/或 `RHO` 设计参数
 - 可通过 `lower_scale / upper_scale / include_e / include_rho` 控制范围和类型
- `POST /import/op2/sensitivity/preview`
- `POST /import/op2/sensitivity/store`

补充说明:

- 这两个接口除了原来的 `op2_path`, 现在还支持可选参数 `matrix_path`。
- 当 `matrix_path` 传入时, 接口会优先使用该结果文件提取灵敏度矩阵; 如果 `matrix_path` 本身就是 `.op2`, 则仍按 OP2 处理。
- 这让接口可以兼容 BAS/FEMTools 风格的矩阵结果文件, 不再只能盯着 `op2`。
- 当前 `matrix_path` 的非 OP2 解析是“纯数字文本矩阵”的第一版启发式解析, 因此返回里会带 `source/source_kind/warnings`, 方便核对到底用了哪个文件、按什么方式读的。

请求示例补充：

```
{
  "project_id": 1002,
  "batch_no": "1",
  "case_name": "nastran_sol200_case_01",
  "matrix_path": "D:/Workspace/Temp/Nastran/UNIT11",
  "bdf_path": "D:/Workspace/Temp/Nastran/227.bdf",
  "metadata_json": "D:/Workspace/Temp/Nastran/227_sol200.bdf.sol200.json"
}
```

2026-05-11 补充 (SOL200 单元级 E 灵敏度)

- `POST /solver/nastran/sol200/preview`
- `POST /solver/nastran/sol200/generate`
- `POST /solver/nastran/sol200/run`

补充说明：

- 现在新增参数预设： `parameter_preset.preset = "all_elements_e"`。
- 这个预设的含义不是“按材料卡生成参数”，而是“先把每个单元局部化成独立的属性/材料引用，再为每个单元生成一个独立 E 参数”。
- 因此：
 - 如果模型有 1548 个单元
 - 最终会生成 1548 个 E 设计变量
- 该预设会在生成结果里额外返回：
 - `parameter_preset_info.localized_input_bdf`
 - `parameter_preset_info.localized_element_count`
 - `generated_files.localized_input_bdf`
- 为了避免 Nastran 在格式化灵敏度 CSV 里截断设计变量标签，单元级 E 参数会使用短标签，例如：
 - E1
 - E2
 - E1548
- 真正的单元对应关系仍然记录在 `metadata_json` 里，通过 `element_id` 追踪，不会丢失
- 这样做的原因是：
 - Nastran 里的 E 通常挂在材料卡上
 - 不是直接挂在单元上
 - 所以如果想让“每个单元一个 E”，必须先把单元各自用的材料/属性拆开
- `POST /import/op2/sensitivity/preview`
- `POST /import/op2/sensitivity/store`
- `POST /export/op2/sensitivity/vtu`

补充说明：

- 灵敏度预览和入库现在支持直接读取 Nastran 的格式化灵敏度 CSV，例如：
 - `sens.csv`
 - `sol200_sens.csv`
- 对这种 CSV，后端会优先解析真正的频率响应块（EIGN），并提取：
 - 响应名
 - 参数名
 - 灵敏度矩阵
- VTU 导出现在支持按 `element_id` 直接映射单元灵敏度。
- 也就是说，当使用 `all_elements_e` 预设时，VTU 云图会按“每个单元自己的参数值”上色，而不是再回退成“按共享材料卡”上色。
-

2026-05-13 接口补充

POST /correlation/modal

接口含义：查询模态相关性矩阵结果。

功能说明：读取 `t_mt_py_fem_modal_correlation` 中已保存的 MAC 结果，并按“图表/热力图友好”的格式返回。矩阵的行是 FEM 模态阶次，列是试验模态阶次。返回结构风格对齐灵敏度矩阵接口。

影响数据库：

- `t_mt_py_fem_modal_correlation`：读，返回当前项目已保存的 MAC 矩阵。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 1002
}
```

2026-05-14 补充： POST /optimization/response/create

接口说明补充：

- 这组接口是给 DSA 灵敏度 `inp` 生成链正式使用的，数据直接写入 `t_mt_py_fem_design_response_catalog`。
- 以后：
 - `POST /sensitivity/dsa/config/preview`
 - `POST /sensitivity/dsa/inp/generate`都直接从这张表读取响应，不再依赖旧的手工响应表。

- 支持两种输入模式：
 - 传 `set_name`：基于已有集合创建响应
 - 不传 `set_name`，改传 `node_labels` 或 `element_labels`：后端自动创建逻辑响应集合，并在生成 DSA `inp` 时写成真实 `NSET/ELSET`
- 约束规则：
 - `region_type=NODE` 时，只能传 `set_name` 或 `node_labels`
 - `region_type=ELEMENT` 时，只能传 `set_name` 或 `element_labels`
 - `set_name` 和标签列表必须二选一

请求字段：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>region_type</code>	<code>string</code>	是	<code>NODE</code> 或 <code>ELEMENT</code>
<code>variables</code>	<code>string[]</code>	是	响应变量，例如 <code>["U2"]</code> 、 <code>["S11"]</code>
<code>set_name</code>	<code>string</code>	否	已存在集合名；与标签列表二选一
<code>node_labels</code>	<code>int[]</code>	否	手工节点号列表；仅 <code>NODE</code> 可用
<code>element_labels</code>	<code>int[]</code>	否	手工单元号列表；仅 <code>ELEMENT</code> 可用
<code>step_name</code>	<code>string</code>	否	分析步名
<code>frequency</code>	<code>int</code>	否	响应频次，默认 1
<code>response_name</code>	<code>string</code>	否	便于前端识别的响应名

已有集合示例：

```
{
  "project_id": 1003,
  "region_type": "NODE",
  "set_name": "RESP_NODES",
  "variables": ["U2"],
  "step_name": "Step-1",
  "frequency": 1,
  "response_name": "DISP_U2"
}
```

手工节点列表示例：

```
{
  "project_id": 1003,
  "region_type": "NODE",
  "node_labels": [10, 20, 30],
  "variables": ["U2"],
  "step_name": "Step-1",
  "frequency": 1,
  "response_name": "DISP_U2_LOCAL"
}
```

手工单元列表示例：

```
{
  "project_id": 1003,
  "region_type": "ELEMENT",
  "element_labels": [115, 120, 121],
  "variables": ["s11"],
  "step_name": "Step-1",
  "frequency": 1,
  "response_name": "STRESS_S11_LOCAL"
}
```

POST /optimization/response

功能说明：

- 按 `project_id` 查询当前项目已配置的正式设计响应列表
- 返回内容里会带：
 - `response_name`
 - `region_type`
 - `set_name`
 - `variables`
 - 若为后台自动创建集合，还会带 `node_labels` 或 `element_labels`

请求示例：

```
{
  "project_id": 1003
}
```

POST /optimization/response/clear

功能说明：

- 清空当前项目的正式设计响应目录
- 用于重新配置响应后再生成新的 DSA `inp`

请求示例：

```
{
  "project_id": 1003
}
```

返回 data 示例:

```
{
  "project_id": 1002,
  "row_mode_order": ["7", "8"],
  "column_mode_order": ["1", "2"],
  "data": {
    "rows": ["7", "8"],
    "column": ["1", "2"],
    "data": [
      [0, 0, 0.973],
      [0, 1, 0.081],
      [1, 0, 0.102],
      [1, 1, 0.956]
    ]
  },
  "summary": {
    "fem_mode_count": 2,
    "test_mode_count": 2,
    "point_count": 4
  }
}
```

POST /correlation/modal/matrix

接口含义: 查询模态相关性矩阵结果 (单独矩阵接口)。

功能说明: 与 POST /correlation/modal 一致, 保留这个单独接口是为了方便前端显式按“热力图数据源”接线。

影响数据库:

- `t_mt_py_fem_modal_correlation`: 读, 返回当前项目已保存的 MAC 矩阵。

请求参数:

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例:

```
{
  "project_id": 1002
}
```

返回 data 示例:

```
{
  "project_id": 1002,
```



```
"row_mode_order": ["7", "8"],
"column_mode_order": ["1", "2"],
"data": {
  "rows": ["7", "8"],
  "column": ["1", "2"],
  "data": [
    [0, 0, 0.973],
    [0, 1, 0.081],
    [1, 0, 0.102],
    [1, 1, 0.956]
  ]
},
"summary": {
  "fem_mode_count": 2,
  "test_mode_count": 2,
  "point_count": 4
}
}
```

POST /correlation/modal/table

接口含义：查询模态相关性表格结果。

功能说明：读取 `t_mt_py_fem_modal_correlation` 中已保存的 MAC 结果，并按“表格友好”的格式返回。返回结构风格对齐灵敏度表格接口。

影响数据库：

- `t_mt_py_fem_modal_correlation`：读，返回当前项目已保存的 MAC 表格数据。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

请求示例：

```
{
  "project_id": 1002
}
```

返回 data 示例：

```
{
  "project_id": 1002,
  "row_mode_order": ["7", "8"],
  "column_mode_order": ["1", "2"],
  "rows": ["7", "8"],
  "column": ["1", "2"],
  "data": [
    {
      "1": 0.973,
      "2": 0.081
    },
  ],
}
```

```
{
  "1": 0.102,
  "2": 0.956
},
"summary": {
  "fem_mode_count": 2,
  "test_mode_count": 2,
  "point_count": 4
}
}
```

POST /import/op2/sensitivity/preview

补充说明：

- `metadata_json` 现在不是前端主流程里的必填项。
- 如果请求里传了 `project_id + batch_no`，后端会优先从 MySQL 里恢复这一次 `sol200` 运行阶段已经保存的：
 - 响应行定义
 - 参数列定义
 - 结果来源路径
- `metadata_json` 保留为兼容兜底参数；如果数据库里没有过程量，仍然可以继续传文件路径。

请求示例：

```
{
  "project_id": 1002,
  "batch_no": "1",
  "matrix_path": "D:/WorkSpace/OtherProjects/VirtualReal702/run/sol200_sens.csv",
  "bdf_path": "D:/WorkSpace/OtherProjects/VirtualReal702/run/fem15_py.bdf"
}
```

2026-05-14 补充：DSA 参数/响应与 `inp` 生成链

POST /optimization/parameter/create

补充说明：

- 当前接口既支持基于已有 `set_name` 创建参数，也支持直接传 `element_labels`。
- 当传入 `element_labels` 时，后端会自动创建逻辑参数集合；后续生成 DSA `inp` 时，再把它写成真实 `ELSET`。

影响数据表：

- `t_mt_py_fem_quantity_set_capability`：读，校验该物理量是否允许创建参数，并尝试自动解析当前值。
- `t_mt_py_fem_selected_parameter`：写，保存正式优化参数。
- `t_mt_py_console_log`：写，记录参数创建过程摘要。

POST /optimization/response/create

补充说明：

- 这是 DSA 正式响应配置接口。
- 支持两种模式：
 - 传 `set_name`：使用已有集合
 - 传 `node_labels` / `element_labels`：由后端自动创建逻辑响应集合
- 后续：
 - `POST /sensitivity/dsa/config/preview`
 - `POST /sensitivity/dsa/inp/generate`都直接从正式响应目录读取。

影响数据表：

- `t_mt_py_fem_design_response_catalog`：写，保存正式设计响应目录。
- `t_mt_py_console_log`：写，记录响应创建摘要。

POST /optimization/response

补充说明：

- 查询当前项目已配置的正式设计响应列表。

影响数据表：

- `t_mt_py_fem_design_response_catalog`：读，返回正式设计响应目录。

POST /optimization/response/clear

补充说明：

- 清空当前项目的正式设计响应目录，便于重新配置后再生成新的 DSA `inp`。

影响数据表：

- `t_mt_py_fem_design_response_catalog`：删，清空当前项目正式设计响应。
- `t_mt_py_console_log`：写，记录清空摘要。

POST /sensitivity/dsa/config/preview

接口含义：从数据库中读取“正式参数 + 正式响应”，预览即将生成的 DSA 配置，不真正写 `inp` 文件。

功能说明：

- 用来先判断“数据库输入对不对”，再判断“生成器写得对不对”。
- 返回：
 - `config_json`
 - `parameter_count`
 - `response_count`
 - `warnings`

- 如果响应没有现成 `set`，但数据库里保存了 `node_labels` / `element_labels`，预览会把它整理成后续生成器可直接使用的 `nodes/elements` 形式。

影响数据表：

- `t_mt_py_fem_selected_parameter`：读，读取正式优化参数。
- `t_mt_py_fem_design_response_catalog`：读，读取正式设计响应目录。
- `t_mt_py_fem_quantity_set_capability`：读，补参数单元信息与当前值信息。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>value_mode</code>	<code>string</code>	否	<code>inherit</code> 或 <code>explicit</code> ，默认 <code>inherit</code>

请求示例：

```
{
  "project_id": 1003,
  "value_mode": "inherit"
}
```

POST /sensitivity/dsa/inp/generate

接口含义：从数据库中读取“正式参数 + 正式响应”，生成 DSA 灵敏度求解 `inp`。

功能说明：

- 先复用 `/sensitivity/dsa/config/preview` 的结果做完整性检查。
- 然后生成：
 - 主分析 `inp`
 - `include.inp`
 - 配置快照文件
- 如果参数或响应是后台自动创建的逻辑集合，生成器会在 `include.inp` 中写出真正的 `ELSET/NSET`。

影响数据表：

- `t_mt_py_fem_selected_parameter`：读，读取正式优化参数。
- `t_mt_py_fem_design_response_catalog`：读，读取正式设计响应目录。
- `t_mt_py_fem_quantity_set_capability`：读，补参数集合信息。
- `t_mt_py_console_log`：可能写，记录生成摘要或警告信息（若后续流程接入）。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

参数名	类型	必填	说明
input_inp	string	是	基础模型 inp 路径
output_dir	string	否	输出目录
main_output	string	否	主输出 inp 路径
include_file	string	否	include 文件名或路径
config_file	string	否	配置快照文件路径
value_mode	string	否	inherit 或 explicit, 默认 inherit

请求示例:

```
{
  "project_id": 1003,
  "input_inp": "D:/WorkSpace/OtherProjects/VirtualReal702/run/base_model.inp",
  "output_dir": "D:/WorkSpace/OtherProjects/VirtualReal702/run/dsa_case_01",
  "value_mode": "inherit"
}
```

POST /import/op2/sensitivity/store

补充说明:

- metadata_json 现在不是前端主流程里的必填项。
- 如果请求里传了 project_id + batch_no, 后端会优先从 MySQL 里恢复这一批次对应的参数列与响应行元数据, 再完成灵敏度入库。
- 推荐前端主流程只维护:
 - project_id
 - batch_no
 - matrix_path/op2_path
 - bdf_path

请求示例:

```
{
  "project_id": 1002,
  "batch_no": "1",
  "case_name": "fem15_all_eleme",
  "matrix_path": "D:/WorkSpace/OtherProjects/VirtualReal702/run/sol200_sens.csv",
  "bdf_path": "D:/WorkSpace/OtherProjects/VirtualReal702/run/fem15_py.bdf",
  "async_submit": false
}
```

POST /export/op2/sensitivity/vtu

补充说明：

- `metadata_json` 现在不是前端主流程里的必填项。
- 如果请求里传了 `project_id + batch_no`，后端会优先从 MySQL 中读取存储好的参数元数据，再将单元级灵敏度映射成 VTU 云图。
- 只有当数据库里不存在该批次过程量时，才建议再回退使用 `metadata_json`。

请求示例：

```
{
  "project_id": 1002,
  "batch_no": "1",
  "input_bdf": "D:/Workspace/OtherProjects/VirtualReal702/run/fem15_py.bdf",
  "output_vtu":
  "D:/Workspace/OtherProjects/VirtualReal702/run/fem15_freq1_all_elem_e.vtu",
  "response_name": "FREQ1"
}
```

2026-05-14 补充： POST /optimization/parameter/create

接口说明补充：

- 参数创建现在支持两种输入模式：
 - 传 `set_name`：基于已导入的 INP 目录中现有集合创建参数。
 - 传 `element_labels`：基于手工单元号列表创建参数，后端会自动生成逻辑 set，并在后续 DSA inp 生成时把它写成真实 ELSET。
- `set_name` 和 `element_labels` 必须二选一，不能同时传入。
- 当传入 `element_labels` 时，建议同时传 `current_value`，否则后续 `explicit` 模式无法稳定生成可算的 DSA inp。
- 当 `selection_mode=LOCAL` 且传入 `element_labels` 时，后端会自动展开成“一单元一参数”。
- 当 `selection_mode=GLOBAL` 且传入 `element_labels` 时，这批单元共享一个参数。

新增请求字段：

参数名	类型	必填	说明
<code>element_labels</code>	<code>int[]</code>	否	手工指定的单元号列表；与 <code>set_name</code> 二选一
<code>current_value</code>	<code>float</code>	否	当前参数值；传 <code>element_labels</code> 时建议填写

手工单元 LOCAL 示例：

```
{
  "project_id": 101,
  "quantity_code": "H",
  "selection_mode": "LOCAL",
  "parameter_name": "T_LOCAL_TEST",
  "element_labels": [101, 102, 103],
  "current_value": 2.5,
  "lower": 1.0,
  "upper": 3.0
}
```

手工单元 GLOBAL 示例:

```
{
  "project_id": 101,
  "quantity_code": "H",
  "selection_mode": "GLOBAL",
  "parameter_name": "T_GLOBAL_TEST",
  "element_labels": [201, 202, 203],
  "current_value": 2.5,
  "lower": 1.0,
  "upper": 3.0
}
```

2026-05-15 补充: POST

/solver/nastran/sol103/run_and_store_modal

接口含义:

给一份 bdf, 自动完成 SOL103 求解, 并继续调用系统现有的 OP2 模态结果导入 流程, 把模态结果写入数据库。

功能说明:

- 这是一条一步式链路:
 - BDF -> SOL103 -> Nastran 求解 -> OP2 -> 模态结果入库
- 当前“上传结果”走的是系统现有的模态导入能力, 本质上等价于:
 - 先调用 POST /solver/nastran/sol103/run
 - 再调用 POST /import/op2/modal/store
- 适合快速验证:
 - bdf 是否能稳定产出 op2
 - op2 是否能被系统正确导入

影响数据表:

- t_mt_py_fem_modal_result: 写, 写入有限元模态结果。
- t_mt_py_fem_modal_correlation: 删 (当 overwrite=true 时), 清空旧模态相关性结果, 避免和新模态结果混用。

请求参数:

参数名	类型	必填	说明
project_id	int	是	项目 ID
input_bdf	string	是	基础 BDF 路径
output_bdf	string	否	生成后的 SOL103 BDF 路径
settings	object	否	SOL103 求解控制参数
nastran	string	否	Nastran 可执行命令
timeout_sec	int	否	求解超时秒数
extra_args	string[]	否	额外求解参数
overwrite	bool	否	是否覆盖导入该项目旧模态结果，默认 true
subcase_id	int	否	指定导入的子工况
mode_numbers	int[]	否	指定导入的模态阶次
instance_name	string	否	导入时写入的实例名
part_name	string	否	导入时写入的部件名
async_submit	bool	否	是否后台提交

请求示例：

```
{
  "project_id": 1002,
  "input_bdf": "D:/Workspace/OtherProjects/VirtualReal702/run/demo.bdf",
  "output_bdf": "D:/Workspace/OtherProjects/VirtualReal702/run/demo_sol103.bdf",
  "settings": {
    "dynamic.fmin": 0.0,
    "dynamic.fmax": 1000.0,
    "dynamic.vectors": 20,
    "result.target": "OP2",
    "embed_geometry": true
  },
  "overwrite": true,
  "mode_numbers": [1, 2, 3],
  "async_submit": false
}
```

settings 补充说明：

- result.target：控制结果输出目标，现有支持 OP2、F06、BOTH
- embed_geometry：是否希望生成的 OP2 带嵌入几何模型
- 当 result.target 为 OP2 或 BOTH 时：
 - embed_geometry=true：生成 PARAM POST -1
 - embed_geometry=false：生成 PARAM POST -2
- 当 result.target=F06 时，不生成 OP2，此时 embed_geometry 不起作用

- 如果显式传入 `settings.post`，则仍以 `post` 为最高优先级，覆盖 `embed_geometry`

2026-05-15 补充: POST

/solver/abaqus/inp/run_and_upload_result

接口含义:

给一份基础 `inp`，直接调用 Abaqus 计算出 `odb`，然后继续调用系统现有的“项目结果上传接口”，把结果上传进项目结果组。

功能说明:

- 这是一条一步式链路：
 - `INP -> Abaqus 求解 -> ODB -> 项目结果组上传`
- 当前“上传结果”走的是系统现有的项目结果组接口，本质上等价于：
 - 先本地运行 Abaqus
 - 再调用 `/api/projects/{project_id}/results`
- 这个接口主要解决“先手动算，再手动上传”太碎的问题。

影响数据表:

- 本服务自身数据库: 不直接写业务结果表。
- 外部项目结果系统: 会新增/更新对应 `project_id` 下的 `result_group` 记录，并触发解析。

请求参数:

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>input_inp</code>	<code>string</code>	是	基础 INP 路径
<code>output_dir</code>	<code>string</code>	否	求解输出目录
<code>abaqus</code>	<code>string</code>	否	Abaqus 可执行命令
<code>job_name</code>	<code>string</code>	否	作业名
<code>cpus</code>	<code>int</code>	否	CPU 数
<code>interactive</code>	<code>bool</code>	否	是否交互式运行，默认 <code>true</code>
<code>timeout_sec</code>	<code>int</code>	否	求解超时秒数
<code>extra_args</code>	<code>string[]</code>	否	额外 Abaqus 参数
<code>result_group</code>	<code>string</code>	否	上传后的结果组名称；不传则自动生成
<code>display_name</code>	<code>string</code>	否	结果组显示名
<code>base_url</code>	<code>string</code>	否	项目结果系统地址
<code>step</code>	<code>string</code>	否	上传结果时限定的 Step
<code>frame</code>	<code>int</code>	否	上传结果时限定的 Frame

参数名	类型	必填	说明
field_prefix	string	否	上传结果时限定的字段前缀
upload_timeout	int	否	调项目结果系统接口的超时秒数，默认 60
wait_timeout_sec	int	否	等待结果组解析完成的超时秒数，默认 3600
poll_interval_sec	float	否	轮询间隔秒数，默认 2.0
async_submit	bool	否	是否后台提交

请求示例：

```
{
  "project_id": 1003,
  "input_inp": "D:/Workspace/OtherProjects/VirtualReal702/run/static_model.inp",
  "output_dir": "D:/Workspace/OtherProjects/VirtualReal702/run/abaqus_case_01",
  "job_name": "static_case_01",
  "result_group": "static_case_01_result",
  "display_name": "static_case_01_result",
  "async_submit": false
}
```

2026-05-18 补充： POST
/correlation/modal/match/preview

接口含义：
基于已经算好的 MAC / DAC / 频率误差结果，给出“候选模态匹配”预览，但不做最终唯一占位。

- 功能说明：
- 这一步主要解决“先看有哪些可能匹配，再决定最终怎么配”的问题。
 - 会先按 MAC 阈值过滤。
 - 如果传了 max_freq_error_ratio，还会再按频率相对误差过滤。
 - 然后对每个分析模态保留前 N 个候选。

影响数据表：

- t_mt_py_fem_modal_correlation：读，读取已经计算好的模态相关性结果。

请求参数：

参数名	类型	必填	说明
project_id	int	是	项目 ID
mac_threshold	float	否	MAC 下限，默认 0.7
max_candidates_per_mode	int	否	每个分析模态最多保留多少候选，默认 3
max_freq_error_ratio	float	否	频率相对误差上限，例如 0.2 表示 20%

请求示例：

```
{
  "project_id": 1002,
  "mac_threshold": 0.7,
  "max_candidates_per_mode": 3,
  "max_freq_error_ratio": 0.2
}
```

返回要点：

- `rows`：候选匹配列表
- `summary`：候选数量、未匹配模态数量
- `unmatched_fem_modes`
- `unmatched_test_modes`

2026-05-18 补充： `POST /correlation/modal/match`

接口含义：

基于已经算好的 `MAC` / `DAC` / 频率误差结果，给出最终唯一模态匹配结果。

功能说明：

- 当前第一版方法支持：
 - `greedy`
- 处理逻辑是：
 - 先按 `MAC` 阈值过滤
 - 再按频率误差阈值过滤
 - 然后按 `MAC` 从大到小做贪心唯一占位
- 一个分析模态最多匹配一个试验模态。
- 一个试验模态最多匹配一个分析模态。

影响数据表：

- `t_mt_py_fem_modal_correlation`：读，读取已经计算好的模态相关性结果。

请求参数：

参数名	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>mac_threshold</code>	<code>float</code>	否	<code>MAC</code> 下限，默认 <code>0.7</code>
<code>max_freq_error_ratio</code>	<code>float</code>	否	频率相对误差上限，例如 <code>0.2</code> 表示 <code>20%</code>
<code>method</code>	<code>string</code>	否	匹配方法，当前支持 <code>greedy</code>

请求示例：

```
{
  "project_id": 1002,
  "mac_threshold": 0.7,
  "max_freq_error_ratio": 0.2,
  "method": "greedy"
}
```

返回要点:

- `rows`: 最终匹配成功的模态对
- `rejected_candidates`: 因为竞争同一个模态而被淘汰的候选
- `summary`
- `unmatched_fem_modes`
- `unmatched_test_modes`

2026-05-19 补充: SOL200 数据库驱动配置接口

- `POST /optimization/sol200/parameter/create`
- `POST /optimization/sol200/parameter`
- `POST /optimization/sol200/parameter/clear`
- `POST /optimization/sol200/response/create`
- `POST /optimization/sol200/response`
- `POST /optimization/sol200/response/clear`

功能说明:

- 这组接口用于正式维护 SOL200 的参数与响应配置。
- 参数不再复用 `inp/DSA` 那套通用响应目录, 响应采用独立配置表。
- 后续 `POST /solver/nastran/sol200/preview`、`POST /solver/nastran/sol200/generate`、`POST /solver/nastran/sol200/run` 在请求体没有显式传 `parameters/responses` 时, 会优先按 `project_id` 从数据库读取这些配置。

影响数据表:

- `t_mt_py_fem_sol200_parameter_config`: 写/读/删, 维护 SOL200 设计参数。
- `t_mt_py_fem_sol200_response_config`: 写/读/删, 维护 SOL200 设计响应。
- `t_mt_py_fem_analysis_run`、`t_mt_py_fem_response_def`、`t_mt_py_fem_parameter_def`、`t_mt_py_fem_sensitivity_result`: 后续 SOL200 灵敏度入库链仍按原流程使用。

请求参数说明:

`POST /optimization/sol200/parameter/create`

字段	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID

字段	类型	必填	说明
<code>parameter_name</code>	<code>string</code>	是	参数名
<code>parameter_type</code>	<code>string</code>	是	参数类型，第一阶段支持 <code>E</code> 、 <code>RHO</code> 、 <code>H/T</code>
<code>initial</code>	<code>float</code>	是	初始值
<code>lower</code>	<code>float</code>	否	下界，缺省时使用 <code>initial</code>
<code>upper</code>	<code>float</code>	否	上界，缺省时使用 <code>initial</code>
<code>property_id</code>	<code>int</code>	否	目标属性 ID
<code>material_id</code>	<code>int</code>	否	目标材料 ID
<code>element_id</code>	<code>int</code>	否	目标单元 ID
<code>extra_json</code>	<code>object</code>	否	扩展信息

POST `/optimization/sol200/response/create`

字段	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID
<code>response_name</code>	<code>string</code>	是	响应名
<code>response_type</code>	<code>string</code>	是	响应类型，第一阶段支持 <code>FREQ</code>
<code>mode_number</code>	<code>int</code>	条件必填	<code>FREQ</code> 响应对应的模态阶次
<code>extra_json</code>	<code>object</code>	否	扩展信息

补充说明：

- `POST /solver/nastran/sol200/preview` 现在支持可选 `project_id`。
- 当 `project_id` 存在且请求体里未传 `parameters`、`responses` 时，服务端会从上述两张配置表自动加载 `SOL200` 配置。
- 如果请求体显式传了 `parameters` 或 `responses`，则优先使用请求体内容，兼容旧用法。

2026-05-19 补充：PBS 远程求解接口

- `POST /solver/pbs/abaqus/run`
- `POST /solver/pbs/nastran/run`
- `POST /solver/pbs/job/status`

功能说明：

- 这组接口用于通过 Altair PBS 远程提交 Abaqus / Nastran 任务。
- 调用 PBS 相关接口前，后端会先检查工程 `project_config.pbs`；如果未配置，则直接报错：请先配置高性能集群计算配置
- 后端按以下链路执行：

- 登录 PBS
- 展开远程 stage 目录
- 创建远程任务目录
- 上传主输入文件
- 校验远程文件是否存在
- 按应用类型组装提交参数
- 提交任务
- 可选轮询任务状态
- 可选列出并下载结果文件
- 配置来源拆分为两部分：
 - `service_config.json` 中的 `PBS.env`、`PBS.environments` 与显式 `PBS.api_paths`
 - 工程配置 `project_config.pbs`
- 其中：
 - `service_config.json` 只负责 PBS 环境地址、账号、stage 路径模板以及各个接口路径；接口路径要直接完整列出，不再通过 `service_prefix/storage_prefix/auth_path` 之类字段拼装
- `project_config.pbs` 负责应用平台信息，例如 `ApplicationId`、`ApplicationName`、`VERSION`、`PLATFORM`，以及按需覆盖 `CORES/HOSTS/PRECISION`

影响数据表：

- 无。
- 该组接口当前只负责任务调度与结果文件下载，不直接写业务库。

`POST /solver/pbs/abaqus/run`

请求字段：

字段	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID；用于读取工程 <code>project_config.pbs</code>
<code>env</code>	<code>string</code>	否	PBS 环境名；未传时使用配置默认值
<code>input_file</code>	<code>string</code>	是	本地 Abaqus 输入文件路径，第一版要求为 <code>.inp</code>
<code>job_name</code>	<code>string</code>	否	PBS 作业名；缺省取输入文件名
<code>output_dir</code>	<code>string</code>	否	本地结果下载目录；未传时默认下载到输入文件同目录
<code>wait</code>	<code>bool</code>	否	是否等待远程任务完成，默认 <code>true</code>
<code>download_results</code>	<code>bool</code>	否	是否下载结果文件，默认 <code>true</code>
<code>poll_interval_sec</code>	<code>float</code>	否	轮询间隔秒数，默认 <code>10.0</code>
<code>wait_timeout_sec</code>	<code>int</code>	否	等待远程任务完成的超时秒数，默认 <code>3600</code>

字段	类型	必填	说明
<code>timeout_sec</code>	<code>int</code>	否	单次 HTTP 请求超时秒数，默认 60
<code>submit_overrides</code>	<code>object</code>	否	提交参数覆盖项，例如覆盖 <code>CORES/HOSTS/PRECISION</code>
<code>async_submit</code>	<code>bool</code>	否	是否后台提交，默认 <code>false</code>

返回要点：

- `job_id`
- `remote_job_dir`
- `remote_primary_file`
- `job_status`
- `resolved_job_state`
- `result_files`
- `result_file_paths`
- `downloaded_files`

工程 `project_config.pbs` 至少应提供：

- `env`：可选，默认 PBS 环境名
- `ApplicationId`
- `ApplicationName`
- `VERSION`
- `PLATFORM`
- 可选的 `CORES`、`HOSTS`、`PRECISION`

`POST /solver/pbs/nastran/run`

请求字段与上面一致，但：

- `input_file` 第一版要求为 `.bdf/.dat/.nas`
- `submit_overrides` 常用于覆盖 `CORES`
- 工程 `project_config.pbs` 中至少应提供 `ApplicationId`、`ApplicationName`、`VERSION`、`PLATFORM`，并可选提供 `CORES`

`POST /solver/pbs/job/status`

请求字段：

字段	类型	必填	说明
<code>project_id</code>	<code>int</code>	是	项目 ID；用于读取工程 <code>project_config.pbs</code> 并先校验 PBS 配置

字段	类型	必填	说明
env	string	否	PBS 环境名
job_id	string	是	PBS 任务 ID
timeout_sec	int	否	单次状态查询请求超时秒数，默认 60

返回要点：

- job_id
- job_status
- resolved_job_state

补充说明：

- resolved_job_state 当前按 PBS 常见状态处理：
 - Q：排队中
 - R：运行中
 - C：完成
 - F：失败
- 当 wait=true 且最终状态为 F 时，接口返回失败。
- 第一版暂不处理多 include 文件、断点续传、失败日志自动解析、任务恢复。

2026-05-21 补充： POST /correlation/modal/match/frequency_scatter

接口含义：

基于已经计算好的 MAC / DAC / 频率误差结果，先按最终唯一模态匹配规则生成匹配模态对，再按“图表散点图友好”的格式返回频率匹配点。

功能说明：

- 该接口复用 POST /correlation/modal/match 的最终唯一匹配逻辑，不返回候选集，而是只返回最终匹配成功的模态对。
- 横坐标使用计算模态频率 freq_fem。
- 纵坐标使用试验模态频率 freq_test。
- 每个散点代表一个最终匹配成功的模态对。
- 返回结构风格对齐现有图表折线接口：
 - data[0].label
 - data[0].xaxis
 - data[0].data
- 同时额外返回 data[0].points，用于前端 tooltip 展示更丰富的点位明细。

影响数据表：

- t_mt_py_fem_modal_correlation：读，读取已经计算好的模态相关性结果。

请求参数:

参数名	类型	必填	说明
project_id	int	是	项目 ID
mac_threshold	float	否	MAC 下限, 默认 0.7
max_freq_error_ratio	float	否	频率相对误差上限, 例如 0.2 表示 20%
method	string	否	匹配方法, 当前仅支持 greedy

请求示例:

```
{
  "project_id": 1002,
  "mac_threshold": 0.7,
  "max_freq_error_ratio": 0.2,
  "method": "greedy"
}
```

返回说明:

- chart_type: 固定为 scatter
- x_label: 横坐标含义, 当前为 calculated_modal_frequency
- y_label: 纵坐标含义, 当前为 test_modal_frequency
- data[0].label: 当前散点系列名称, 固定为 matched_modes
- data[0].xaxis: 横坐标值列表, 字符串形式, 便于和现有图表折线结构对齐
- data[0].data: 散点数组, 每项是 [freq_fem, freq_test]
- data[0].points: 散点明细数组, 前端可直接用来渲染 tooltip
- rows: 最终唯一匹配结果的原始行数据, 便于前端或调用方复用
- summary.point_count: 实际返回散点数量

返回示例:

```
{
  "project_id": 1002,
  "chart_type": "scatter",
  "method": "greedy",
  "mac_threshold": 0.7,
  "max_freq_error_ratio": 0.2,
  "x_label": "calculated_modal_frequency",
  "y_label": "test_modal_frequency",
  "data": [
    {
      "label": "matched_modes",
      "xaxis": ["31.8", "64.2"],
      "data": [
        [31.8, 32.5],
        [64.2, 63.9]
      ]
    }
  ],
}
```

```

"points": [
  {
    "x": 31.8,
    "y": 32.5,
    "tooltip": {
      "title": "FEA 1 - 31.8000Hz, EMA 3 - 32.5000Hz MAC:97.500",
      "fem_mode_no": 1,
      "test_mode_no": 3,
      "freq_fem": 31.8,
      "freq_test": 32.5,
      "mac": 97.5,
      "freq_error_ratio": -0.0215,
      "status": "matched",
      "recommended": true,
      "flip": false
    }
  },
  {
    "x": 64.2,
    "y": 63.9,
    "tooltip": {
      "title": "FEA 2 - 64.2000Hz, EMA 4 - 63.9000Hz MAC:95.000",
      "fem_mode_no": 2,
      "test_mode_no": 4,
      "freq_fem": 64.2,
      "freq_test": 63.9,
      "mac": 95.0,
      "freq_error_ratio": 0.0047,
      "status": "matched",
      "recommended": true,
      "flip": true
    }
  }
]
},
"summary": {
  "fem_mode_count": 6,
  "test_mode_count": 6,
  "matched_pair_count": 2,
  "candidate_conflict_count": 1,
  "unmatched_fem_mode_count": 4,
  "unmatched_test_mode_count": 4,
  "point_count": 2,
  "tooltip_count": 2
},
"rows": [
  {
    "status": "matched",
    "recommended": true,
    "rank": null,
    "fem_mode_no": 1,
    "test_mode_no": 3,
    "mac": 97.5,
    "freq_fem": 31.8,
    "freq_test": 32.5,

```

```
    "freq_error_ratio": -0.0215,  
    "flip": false  
  }  
],  
"unmatched_fem_modes": [3, 4, 5, 6],  
"unmatched_test_modes": [1, 2, 5, 6]  
}
```